

密级:

保密期限:

北京邮电大学

硕士学位论文



题目: 基于微服务的网上商城系统的设计与实现

学 号: 2018140632

姓 名: 杨文豪

专 业: 软件工程

导 师: 张笑燕

学 院: 计算机学院 (国家示范性软件学院)

2021 年 5 月 28 日

中国·北京

密级： 保密期限：

北京邮电大学

硕士学位论文



题目：基于微服务的网上商城系统的设计与实现

学 号： 2018140632

姓 名： 杨文豪

专 业： 软件工程

导 师： 张笑燕

学 院： 计算机学院（国家示范性软件学院）

2021 年 5 月 28 日

Security Classification: Secrecy period:



BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

Thesis for Master Degree

Title: Design and Implementation of Online Mall
System Based on Micro Service

Student No.: 2018140632

Name: Wenhao Yang

Major: Software Engineering

Supervisor: Xiaoyan Zhang

College: School of Computer Science

(National Pilot Software Engineering School)

May, 28, 2021

基于微服务的网上商城系统的设计与实现

摘 要

随着消费水平的提高和科技的发展,人们对于网上购物的需求也越来越强烈,用户对于系统功能的要求也越来越多。用户量的激增也为系统的负载能力带来了极大的挑战。单一应用架构中所有的功能在同一个系统内实现的架构远远达不到需求,服务器在如此大的访问量的作用下将轻易崩溃,因此要解决单体架构的电商项目随着需求迭代而产生的种种问题。

本论文是为了解决单体架构的电商项目随着需求迭代而产生种种问题,因此使用微服务的架构进行设计,保证业务之间充分解耦,服务之间独立运行,实现拓展性高、高可用与高并发的商城系统。系统功能的单体结构以微服务的形式存在(包括购物车管理服务、中央认证服务等)。服务之间通过服务注册、服务容错、服务远程调用、服务追踪、分布式事务等进行服务治理,保证各服务之间的运行与使用安全稳定。将系统的服务功能按照角色的使用情况进行划分,同时以用例图的方式使系统功能与使用者一一对应。在对系统进行总体设计后,并对其中几个服务模块通过时序图的方式进行了详细的设计与描述。在具体的实现中还通过其他的分布式技术诸如消息队列、缓存等,保证了系统的并发能力,提高了系统响应时间和可用性。

本文在最后阶段对系统通过功能和性能两方面进行了全面的测试,保证系统功能正常运行使用,同时能够保持较好的性能指标。通过对测试结果的验证确保了系统的可行性。目前系统已经完成了设计与开发,并完成了全部的测试工作,进行了上线。

关键词: 微服务 网上商城 分布式 服务治理

DESIGN AND IMPLEMENTATION OF ONLINE MALL SYSTEM BASED ON MICRO SERVICE

ABSTRACT

As the consumption level improves and science and technology develops, there is a growing need for online shopping and function system. The surge of users has also brought great challenges to the load capacity of the system. In the architecture, all the functions in a single application are achieved in one system, while now cannot satisfy the demand, and the server will easily collapse under the action of such a large traffic volume. Therefore, various problems derived from the demand iteration of the e-commerce project with the monomer architecture should be solved.

This topic is to solve various problems arising from the monomer structure e-commerce project with the demand iteration. Therefore, the architecture of micro-service is used to design, to ensure full decoupling between businesses, independent operation between services, and to realize the mall system with high scalability, high availability and high concurrency. The single architecture of system functions exists in the form of micro-services (including shopping cart management service, central authentication service, etc.). Service governance is carried out among services through service registration, service fault tolerance, service remote invocation, service tracking, distributed transaction, etc., to guarantee the operation and the service secure and steady. The service feature of the system are partitioned according to the usage of characters, and the system functions are corresponding to users one by one in the way of use case diagram. After the overall design of the system, several service modules are designed and described in detail by means of sequence diagram. In the concrete implementation, other distributed technologies such as message queue, cache, etc., guarantee the concurrency ability of the system, improve the reaction speed and the available of the system.

In the last stage of this paper, the author had finished comprehensive test on function and performance to guarantee that the system operates well and maintains

excellent performance indicators. The test results verification the feasibility of the system. Currently, the design and development of the system and all the testing work have been completed and the system has been on-line.

KEY WORDS: Micro Service, Online Store, Distributed, Service Governance

目录

第一章 引言	1
1.1 课题背景	1
1.2 课题任务	2
1.2.1 课题内容	2
1.2.2 本人承担任务	2
1.3 论文结构	2
第二章 相关技术介绍	5
2.1 SpringCloud 介绍	5
2.1.1 GateWay 网关介绍	5
2.1.2 Sleuth 服务追踪介绍	5
2.2 SpringCloud Alibaba 介绍	6
2.2.1 SpringCloud Alibaba-Nacos 注册中心介绍	6
2.2.2 SpringCloud Alibaba-Sentinel 服务容错介绍	6
2.3 搜索引擎 Elasticsearch 介绍	6
2.4 RabbitMQ 介绍	6
2.5 本章小结	7
第三章 系统的需求分析	9
3.1 系统用户角色分析	9
3.1.1 用户	9
3.1.2 管理员	9
3.2 系统功能需求分析	10
3.2.1 商品管理服务	10
3.2.2 仓储服务	10
3.2.3 支付服务	11
3.2.4 秒杀服务	11
3.2.5 搜索服务	11
3.2.6 中央认证服务	11
3.2.7 购物车管理服务	11
3.2.8 订单管理服务	12
3.2.9 用户管理服务	12
3.2.10 优惠管理服务	12
3.3 系统功能的 UML 用例图	12
3.3.1 管理员的 UML 用例图	12
3.3.2 用户的 UML 用例图	14
3.4 系统的非功能需求分析	17
3.5 本章小结	17
第四章 系统的总体设计	19

4.1 系统软件层次架构设计	19
4.2 系统微服务架构设计	20
4.3 系统功能模块设计	22
4.4 系统的数据库设计	23
4.4.1 E-R 图设计	23
4.4.2 数据库表设计	25
4.5 系统的界面设计	35
4.5.1 后台管理系统的界面设计	35
4.5.2 前台展示的界面设计	36
4.5.3 订单查看的界面设计	36
4.5.4 结算页的界面设计	37
4.5.5 商品搜索页的界面设计	38
4.5.6 商品详情页的界面设计	39
4.6 系统接口设计	40
4.7 本章小结	41
第五章 系统主要功能模块的详细设计与实现	43
5.1 微服务基础组件的详细设计与实现	43
5.1.1 服务的注册与发现的设计与实现	43
5.1.2 服务的远程调用的设计与实现	44
5.1.3 服务的熔断降级的设计与实现	45
5.1.4 服务的网关的设计与实现	45
5.2 中央认证服务模块的详细设计与实现	47
5.2.1 单点登录的设计与实现	47
5.2.2 社交登录的设计与实现	48
5.2.3 用户注册的设计与实现	49
5.3 秒杀服务模块的详细设计与实现	51
5.3.1 秒杀活动的创建的设计与实现	52
5.3.2 秒杀下单的设计与实现	53
5.4 订单管理服务模块的详细设计与实现	55
5.4.1 立即下单的设计与实现	55
5.4.2 订单退款的设计与实现	56
5.5 支付服务模块的详细设计与实现	57
5.6 商品管理服务模块的详细设计与实现	58
5.7 本章小结	60
第六章 系统测试	61
6.1 系统测试方法介绍	61
6.2 系统的功能测试	61
6.3 系统的性能测试	64
6.4 测试结果分析	64
6.5 本章小结	65
第七章 结束语	67
7.1 论文工作总结	67

7.2 问题和展望	68
参考文献	69
致 谢	71

第一章 引言

1.1 课题背景

随着社会的发展,日常生活的人们有很少的时间去线下进行购物,人们迫切的需要一种以低时间成本的方式获取满意的商品,人们不再愿意因为商场的距离遥远或者与各种各样商品进行反复挑选导致花费大量的时间。商品的数量琳琅满目,但是符合自己心里预期的商品却没有那么多,人们为了购买一件商品,却需要在茫茫的商品种类中挑选自己满意的商品,这期间将花费大量的时间,造成时间的浪费。同时人们在商场中了解商品的方式只有通过自己的主动观看和导购员的解说。而导购员的引导性解说往往不会让人们对产品有更好,更深入的了解,导购的解说大部分会具有一定的片面性。同时如果想要在不同的地方对多种商品进行比对时,只能通过个人的记忆去进行比对,无法获取直观的比较结果,长此以往消费者往往购买不到自己最满意的商品。此时人们渴望一种随时随地都可以对产品进行挑选对比的方式进行购物,网上商城应运而生。网络环境越来越完善,获取网络资源的速度与容量的大幅提升,标志着新电商时代的到来^[1]。网上购物不仅大大降低了购物的时间成本,人们只需要花费少量的时间就可以了解商品的详细信息。随着网上购物的使用,人们的消费习惯与消费理念也发生了很大的改变,随时随地的购物方式极大的提高了商品的成交量,足不出户成为新的购物体验。同时购物系统本身的保存功能保证了消费者能够轻松对不同的商品进行同时比较,消费者通过多种商品的对比,选择更符合自己心意的产品。同时消费者还可以快速的查询自己想要的商品,对商品进行准确的定位,避免长时间的信息浏览对时间造成浪费。

消费者对于线上购物的需求越来越多,开发者为了满足系统正常的使用情况只能不断地增加自己的业务,然而无休止的增加对线上系统带来了极大的挑战。系统的开发者渴望开发出更多地功能来满足用户购物的需要,此时系统在使用时,系统调用越来越复杂的问题。在目前主流的电商平台的实现方案中,由于社区的活跃、完备的开发者生态和开发敏捷迅速等优点,大量的企业选择 Java 所对应的框架进行系统的开发,在单系统的架构设计无法满足功能正常运行需要的时候,为了采用更好的架构去设计, Spring+SpringMVC+Mybatis 变成了开发选择的主流。但是当系统使用这样的思想对系统进行设计时,虽然提高了代码的复用性,系统完成了解耦,然而对于这样一种架构,每一次功能调用的状态和位置就需要被调用模块完整的记录下来,此时只要位置或者状态一被改变就要更新

所有的配置信息，给系统的开发和运维工作带来了极大的难度，同时这样的系统不利于持续交付^[2]。随着业务的发展，这样的系统也无法支持更快的业务发展^[3]，此时产生了 SOA(Service-Oriented Architecture, 面向服务的架构) 服务化架构^[4]：以应付越来越繁杂的系统^[5]。使系统功能相互之间不再强关联，不仅降低了功能的耦合度，还保证了系统在增加功能时不对原有的功能造成影响。

1.2 课题任务

1.2.1 课题内容

本课题是为了解决单体架构的电商项目随着需求迭代而产生种种问题，因此使用微服务的架构进行设计，保证业务之间充分解耦，服务之间独立运行，实现拓展性高、高可用与高并发的商城系统。系统功能的单体结构以微服务的形式存在（包括购物车管理服务、中央认证服务等）。服务之间通过服务注册、服务容错、服务远程调用、服务追踪、分布式事务等进行服务治理，保证各服务之间的运行与使用安全稳定。通过以不同角色进行对于系统需求的分析，确定系统的功能，在对系统的功能进行开发之后要对系统全面的测试以保证该商城系统的可用性。

1.2.2 本人承担任务

本人主要通过前期的调研，在阅读了相关文献以及了解当前现实存在的问题下，了解了相关系统的痛点以及解决方法。通过对用户与管理员两种角色对于系统使用的实际需求出发，将系统的服务功能划分为商品管理服务、仓储服务、支付服务、秒杀服务、搜索服务、中央认证服务、购物车管理服务、订单管理服务、用户管理服务、优惠管理服务。完成系统的总体设计包括软件层次架构设计、数据库设计、界面设计和接口设计。最后主要针对中央认证服务、秒杀服务、订单管理服务、支付服务和商品管理服务进行详细的设计与实现，编码实现了系统的功能。通过测试用例以及压测工具的使用，保证系统在线上环境的功能可靠性和性能的稳定性。

1.3 论文结构

本文共分七章，内容安排如下：

第一章 引言，本章节主要介绍课题的选题背景、课题任务和本人承担的任务。

第二章，相关技术介绍，本章节主要介绍了该架构设计下所使用的相关技术，为系统的开发与运行提供技术支持。

第三章，系统的需求分析，本章节在前期根据课题背景与用户购买商品的实际行动进行相关调研，在了解了相关方法技术后，通过以不同角色进行对于系统需求的分析，确定系统的功能，再根据角色对服务功能的使用程度，绘制了用户和管理员的 UML（Unified Modeling Language，统一建模语言）用例图，同时对 UML 用例图进行了相关的描述，最后通过系统的性能指标介绍了系统所需要的非功能需求。

第四章，系统的总体设计，本章节主要介绍了系统的总体设计，从系统软件层次架构设计、系统功能模块设计、系统的数据库设计、系统的界面设计和系统的接口设计对系统进行总体设计。

第五章，系统主要功能模块的详细设计与实现，本章节主要从中央认证服务所满足的单点登录、社交登录与注册需求、秒杀服务的活动创建以及秒杀下单需求、订单管理服务的立即下单和订单退款需求、支付服务、商品管理服务等方面进行了介绍。

第六章，系统测试，本章节为了保证系统在线上环境的功能可靠性和性能的稳定性，从系统测试方式、系统的功能测试、系统的性能测试、测试结果分析等几个部分对系统的测试进行描述。

第七章 结束语，本章节主要进行论文的工作总结和待解决问题的描述，以及对其他的未实现目标的展望。

第二章 相关技术介绍

本章主要是对该系统相关的技术介绍，了解微服务架构设计常用的技术手段，保证了系统实现上的技术支持。包括 SpringCloud 和 SpringCloud Alibaba 相关的组件、Elasticsearch 搜索引擎和 RabbitMQ 消息队列。

2.1 SpringCloud 介绍

Spring Cloud^[6]是在 Spring boot 应用开发之上，通过提供诸如服务注册发现等分布式的工具，帮助开发者更快的搭建分布式系统，为系统的开发提供了很大的帮助，极大的缩短了系统搭建的时间。

2.1.1 GateWay 网关介绍

网关是用户访问的系统的第一层，也是系统屏蔽违规访问、防止后台侵入、获取数据的路由、维持系统稳定性的第一道屏障。当用户的请求到达网关层时，网关先判定当前用户的请求是否携带其他违规信息，然后对请求的路径进行校验，判断当前访问的路径是否符合该请求的身份，之后判断该请求是否在短时间内已经请求，避免爬虫软件对系统的多次访问。请求校验完成后，将当前的请求记录到日志中，以便后期有迹可循。SpringCloud gateway^[7]作为新一代的网关技术，通过简单便利的方式将用户的请求路由到指定区域，保证了系统的安全性和稳定性。

2.1.2 Sleuth 服务追踪介绍

微服务架构将传统的单系统中的业务与需求进行划分，保证了各功能之间的调用的关系的同时，其系统功能独立运行。独立的功能只对外提供接口，而不会造成其他功能的良好与否对整个系统的功能造成影响。对于集成的单系统来说，功能之间的发现通过简单的配置，系统之间的调用都在开发阶段进行定义，通过查看源代码的方式就可以对系统的调用关系有很好的认识。而微服务的分布式架构是将功能独立运行在其他服务器，当系统功能出现问题的时候无法通过一台服务器就可以了解到整个链路的调用关系，对于功能的源代码也无法轻易的读取，参与的顺序也无法查找，这对于问题的发现是极为困难的，因此要对服务的调用关系有一个清晰的认识。微服务跟踪^[8](sleuth)就是一种获取调用链路，便于问题定位的工具。通过这种工具，可以清晰地获取服务的调用链路，准确的查看当问题出现时链路的断开位置，极大的提高了问题的反馈与处理时间。

2.2 SpringCloud Alibaba 介绍

对于微服务的开发来说, SpringCloud Alibaba^[9]通过提供一系列的中间件将分布式的搭建变得简单, 在接接入了中间件之后, 只需要一些简单配置信息, 就可以使用中间件来解决微服务中存在的问题, 极大的提高了开发效率。

2.2.1 SpringCloud Alibaba-Nacos 注册中心介绍

由于微服务系统的分布式架构促使服务独立运行在单独的模块, 服务之间的调用关系通过系统的接口进行访问。因此当前运行的服务是否能被发现, 想要查询其他服务的接口时是否能正确获取到所需要的服务信息就变得尤为重要。Nacos^[10] 致力于帮助发现、配置 和管理微服务。通过输入相关信息后, 将当前运行的服务在 Nacos 上进行注册, 注册完成的服务就会保存在列表中。当产生服务调用时, 就去服务的列表中进行查看, 找到当前运行服务之后就可以获取服务运行的接口, 实现功能上的对接。Nacos 可以更敏捷和容易地构建、交付和管理微服务平台。

2.2.2 SpringCloud Alibaba-Sentinel 服务容错介绍

对于微服务的系统来说, 服务的稳定性决定了系统的稳定性, 服务器的宕机决定了运行在当前服务器的功能的不可用性。因此要通过一系列的策略保证服务的高可用性, 极大的降低服务崩溃的可能性。Sentinel^[11]通过对请求访问的控制, 从当前服务的负载出发, 当访问的流量过多时进行过滤、隔离、排队的方式进行流量分割, 保证当前的流量请求在服务的负载之内, 从而保证了服务的稳定性。

2.3 搜索引擎 Elasticsearch 介绍

搜索是一个系统的基本功能, 对于商城系统来说, 大量的商品信息铺面而来, 用户在商品的简单浏览中往往很难找到符合自己心意的商品。当用户心中已经对商品的需求有了大致的范围之后, 通过简单的点击就可以直接获取自己想要的商品。同时当用户进行搜索时, 系统的反馈时间的长短也尤为重要, 避免用户在搜索后进入漫长的等待。

Elasticsearch^[12]可以快速进行全文搜索, 其分布式存储的特点保证了数据的实时分析与及时响应。其内部集群通过分片的方式不仅保证了数据在分布式存储上的响应时间, 同时保证了数据的持久性。

2.4 RabbitMQ 介绍

微服务系统在使用过程中, 服务之间的调用不关心服务的运行方式, 只为了

获取自己想要的数据库。为了保证系统响应时间的快速性，服务之间不直接进行消息的传递。因此当服务产生调用时，当前服务会告知调用服务自己想要什么数据，同时当前服务不会因为未获取到数据而进入忙等待状态，而是会产生一条想要获取数据的消息，服务的被调用者获取了消息之后，通过消息的内容进行相应的操作，完成之后将相应的数据再通过消息的方式发送给调用者服务，服务的被调用者继续去完整其他的消息。RabbitMQ^[13]就是消息中间件，通过内部的机制，保证了消息在传递过程中的准确性与可靠性。

2.5 本章小结

本章主要描述了开发过程中将使用到的技术需求，对于 SpringCloud 和 SpringCloud Alibaba 的相关组件部分，介绍了网关 Gateway、服务追踪 Sleuth、注册中心 Nacos、限流降级 Sentinel，通过相关的微服务组件，对于系统微服务的架构设计起到了至关重要的作用。同时还介绍了分布式系统中较为关键的技术，搜索引擎 Elasticsearch 和 RabbitMQ 消息队列。通过搜索引擎，能够帮助用户更好对商城信息进行检索，找到自己满意的商品。通过消息队列，保证了消息的异步消费，极大的提升了系统的并发能力。

第三章 系统的需求分析

本章节从使用者的角度对该系统的需求进行分析,使用者包括用户和管理员。通过角色要在该系统满足的基本功能为出发点,对系统需求进行详细的分析,用户的需求是购物,因此系统的功能要满足用户的购物需求,而对于管理员来说,通过后台的管理能够使用户获得更好的购物体验。因此用户与管理员功能共同结合才能使系统更好为用户服务。然后通过不同角色所使用功能的不同的用例图能够更好的了解角色与功能之间的关系。最后通过对系统的性能指标介绍了系统所需要的非功能需求。

3.1 系统用户角色分析

3.1.1 用户

用户作为该系统的主要使用者,用户的体验、用户所需要的功能将作为系统需求分析的主要考虑部分。对于用户来说,用户访问系统时会先浏览到系统展示的商品,在用户浏览商品后用户有两种后续的方式,一种是用户选择登录,另一种是对商品进行检索。在选择登录中,用户可以选择系统的账号信息和社交登录进行登录。当用户输入了登录信息之后判断用户账号进行校验。登录成功后用户可以继续浏览商品,此时系统回显用户信息。如果用户选择了搜索商品,此时有三种方式进行商品的检索,分别是输入关键词进行检索、选择分级进行检索、选择筛选条件进行检索,搜索完成后系统会根据用户的选择展示相关的商品。在用户选中自己比较满意的商品后,点击商品会进入商品详情页,用户在查阅了商品的详情后,如果对商品比较满意,用户会选择加入购物车或者直接购买。如果用户选择了直接购买,系统会提示用户选择合适的规格以及商品数量。如果用户加入购物车,那么用户就可以在购物车中修改或者删除商品。如果用户选择了立即结算,那么系统会促进用户支付。如果用户未支付,系统会保留订单的状态,用户本次的订单流程结束,用户可以选择其他时间继续支付。如果用户选择了支付,系统会保存用户的订单信息让用户确认,用户确认完成后会给用户发货。当用户收到货物时,用户会对货物进行检查,如果用户满意,本次购物流程结束,如果不满意,用户会发起退货退款,等到用户收到退款后,本次流程亦结束。

3.1.2 管理员

管理员作为系统后台的管理者,要通过一些后台管理功能帮助用户获得更好的购物体验。对于商品的管理来说,因为商品的种类很多,为了将同类型的商品

整理在一起保证用户的检索和自己对于商品的查找管理，要对商品进行多级分类，多级分类的路径可以用来作为当前访问路径提供给用户。每一个商品都有自己的品牌，一个品牌有多个商品，商品的品牌作为用户经常搜索的关键字以及用户采购的关键选择，要对商品的品牌进行单独的管理。商品同样有其特有的属性，如衣服的尺寸、腰围，数码产品的内存、书籍的图文、童话故事等，每一种商品都会有特有属性。管理员对于新采购的商品要及时地让用户感知，促进用户交易的生成。对于库存不足或者过季的产品要及时下架，保证商品信息的及时性和准确性。管理员还可以根据一些节日时间进行商品的优惠或者一些商品的限时抢购，既可以提高用户活性，也可以提高成单率。用户购买的商品数量是要受商品库存的限制，因此要对商品的库存进行管理，商品库存过量就可以进行优惠活动，商品库存不足就可以进行采购，采购是由专门的采购员进行采购，库存中同样要保留采购清单，方便对采购信息的回看。在用户下单时，要对用户的订单进行维护，订单的状态要根据用户的行为进行状态的流转，同时对于订单要能够进行搜索，包括状态搜索、时间搜索、订单编号搜索等方式，方便管理员对用户的订单进行管理。因此，管理员的需求主要包括对商品的管理，包括品牌、分类、商品上下架、优惠活动、库存信息、订单信息等，对于用户的管理主要是用户的信息进行管理。

3.2 系统功能需求分析

对于该商城系统的功能需求进行分析时，结合角色行为所需要的功能来看，在保障系统能够满足所有需求的前提下，系统的主要功能分为商品管理服务、仓储服务、支付服务、秒杀服务、搜索服务、中央认证服务、购物车管理服务、订单管理服务、用户管理服务、优惠管理服务。

3.2.1 商品管理服务

商品管理服务主要是管理员用来管理商品所进行的一系列操作。如商品分类主要是将商品按不同层级的类别进行分类，便于商品查找。商品分类是以树形结构进行展示，管理员在对分类进行增删改查时需要注意分类之间的父子关系；商品品牌管理主要是对商品的品牌进行统一的维护；商品属性分组时要进行规格参数的细化，分组展示不同的属性，保证商品详情页的正常展示；商品上下架功能主要是对商品进行展示与隐藏，对于新货商品要进行上架展示，而对于过季或者库存不足的商品予以下架。

3.2.2 仓储服务

仓储服务包括仓库管理、库存管理、采购单管理。仓库管理是对所有商品所在的仓库进行管理，对于仓库的信息进行统一维护；商品的库存管理是对当前仓

库所有的商品进行管理,对于商品的数量进行统一维护,当商品数量不足时要进行添加库存,而对商品库存过剩时可采取降价操作;商品采购单的管理是对商品采购信息的维护,保存了当前采购金额、采购商品、采购数量等信息,对于商品采购单的维护是为了后期的核对校验。

3.2.3 支付服务

支付服务是给用户在购买商品时提供支付的功能,帮助用户完成订单中的支付步骤。支付服务保存了当前支付时间、支付方式、支付金额等信息。

3.2.4 秒杀服务

秒杀服务通过以低价购买商品的方式吸引用户下单,以达到提高用户数,增加用户活跃度,提高下单数的目的。秒杀服务包括秒杀活动的创建、秒杀商品的上架、秒杀下单。秒杀活动的创建是将当前秒杀活动填入相关的秒杀时间、标题等进行活动的初始化操作,活动只有被创建后才能关联相关商品;秒杀商品的上架主要是将本次活动的商品与活动相关联,不将商品在活动创建时直接创建是为了活动与商品的复用性;秒杀下单主要是用户进行抢购秒杀商品的行为,秒杀下单与普通商品的下单隔开是因为当前活动短时间内下单很多,防止对其他正常下单造成影响。

3.2.5 搜索服务

搜索服务是用户对自己想要购买商品的特征进行搜索,搜索的方式包括关键词搜索、分级选择搜索、条件筛选搜索。关键词搜索是用户可以通过输入关键词进行定向搜索;分级选择搜索是用户选择商品的分级,通过多层分级进行多层搜索;条件筛选搜索是根据输入条件的不同进行条件的过滤,如价格区间、时间区间等。

3.2.6 中央认证服务

中央认证服务主要是对当前用户状态的查询,用户通过认证后才能使用更多的功能,包括单点登录、用户注册、社交登录。单点登录是用户只通过一次登录就可在跳转所有界面时携带个人信息,提高了系统的响应时间;用户注册是提供用户注册界面,保存用户的个人信息;社交登录是用户可以使用第三方软件进行登录,增加了用户登录的渠道。

3.2.7 购物车管理服务

购物车管理服务是对用户挑选出的未购买货物的维护,包括添加到购物车、购物车商品数量修改、删除购物车内商品、购物车商品合并。商品添加之后可以查看到自己所选的商品;购物车商品数量修改是在购物车中修改购买单品的数量;删除购物车内商品是将购物车中的商品删除;如果货物在未登录时被添加,那么在登录之后可以将之前的商品与当前购物车商品进行合并,保证了用户在未

登录时添加购物车功能的可用性。

3.2.8 订单管理服务

订单管理服务是电商系统的枢纽,在订单这个环节上需求获取多个模块的数据和信息,同时对这些信息进行加工处理后流向下个环节,这一系列就构成了订单的信息流通。订单的管理功能包括立即下单、订单查看、订单删除、订单退款。立即下单是在用户在购买商品时点击立即下单后,系统创建订单保存了当前订单的创建时间、订单金额等信息;订单可以被查看和删除;订单退款是用户发起退款操作时所需要的功能。

3.2.9 用户管理服务

用户管理服务是用户对个人信息的维护以及管理员对用户信息的维护,用户管理功能包括基本信息管理、收货信息管理、个人订单管理、会员信息管理。基本信息管理是用户对自己的个人基础信息进行管理;收货信息管理主要是用户对于自己的收货信息进行管理;个人订单管理是用户对自己的订单进行搜索删除等操作。会员信息管理是管理员保持对系统注册用户基本信息的管理,包括违规用户的禁用与启用、用户密码重置。违规的用户主要是用户存在刷单的行为,用户密码重置主要是防止用户忘记密码而无法登录系统。

3.2.10 优惠管理服务

优惠管理服务只要是对商品的优惠相关信息进行管理,包括优惠活动记录和优惠券管理。优惠活动记录是保存了当前已经完成的活动信息,便于进行活动的比对;优惠券管理可以对优惠券进行创建与删除,保存了当前优惠券的优惠金额、使用条件等信息。

3.3 系统功能的 UML 用例图

下面使用 UML 用例图的方式来更好的表示角色与系统功能之间的关系。

3.3.1 管理员的 UML 用例图

该系统的管理员主要来管理商品以协助用户获得更好的购物体验,满足用户对于系统所需要的基本功能。系统的管理员功能包括商品管理服务、仓储服务、订单管理服务、秒杀服务、优惠管理服务。管理员通过商品管理服务,可以对商品进行分类;管理员对每一个商品的品牌进行管理,保证用户可以将品牌作为关键词进行商品的检索;商品的属性分为 SPU (Standard Product Unit, 标准化产品 单元) 属性和 SKU (Stock Keeping Unit, 库存量单位) 属性,SKU 属性是商品的一种类型属性,他是一种商品类型的抽象,如手机就是一种 SPU,其属性包括颜色、内存大小等,而对于 SKU 属性来说是一件具体的商品,如白色的 4G 内存手机,就是一个 SKU。管理员通过对这种属性进行分组,能够更好的管理

商品的属性,对于商品详情页的展示来说可以获得更好的分块,让用户对于商品的基础属性和特有属性有更好的了解,便于用户的选择;同时管理员还可以对商品进行上下架的操作,对于新品来说要及时的上架,便于用户及时的购买,而对于过期的或者库存不足的商品要及时地进行下架,对商品要做到及时的更新,避免用户购买到不满意的商品。管理员的仓储服务包括仓库管理、商品库存管理和采购单的管理。仓库管理主要是管理当前的仓库信息;商品的库存管理主要是对商品所在的仓库以及商品的数量进行管理;管理员通过仓储管理,可以对仓库所在的位置进行记录,同时在仓库中记录下当前所有的货物的详细数量信息,通过查看这个货物的数量,了解商品的售卖情况,对于打折或者再次购物起到指引的作用。当进行货物的清单发现商品不足时要进行采购,同时要对这些信息进行详细的记录,做到采购的信息要有迹可循,保证商品在入库时是完好的,也可以作为后期盘点的主要证据。订单管理中,包括订单的查看和删除。管理员的秒杀服务主要包括秒杀活动的创建和秒杀商品的上架,只有管理员设置了相关的活动,用户才可以参与。优惠管理服务主要保存了一些优惠券信息以及用户参与过的优惠活动记录,用户购买商品的时候首先查看自己是否有相关的优惠卷,当有优惠券使用时可以在付款时使用优惠卷抵消部分付款金额。管理员的用户管理服务主要是会员信息管理,促进了用户对系统使用的良好性与规范性。管理员的 UML 用例图如 3-1 所示。

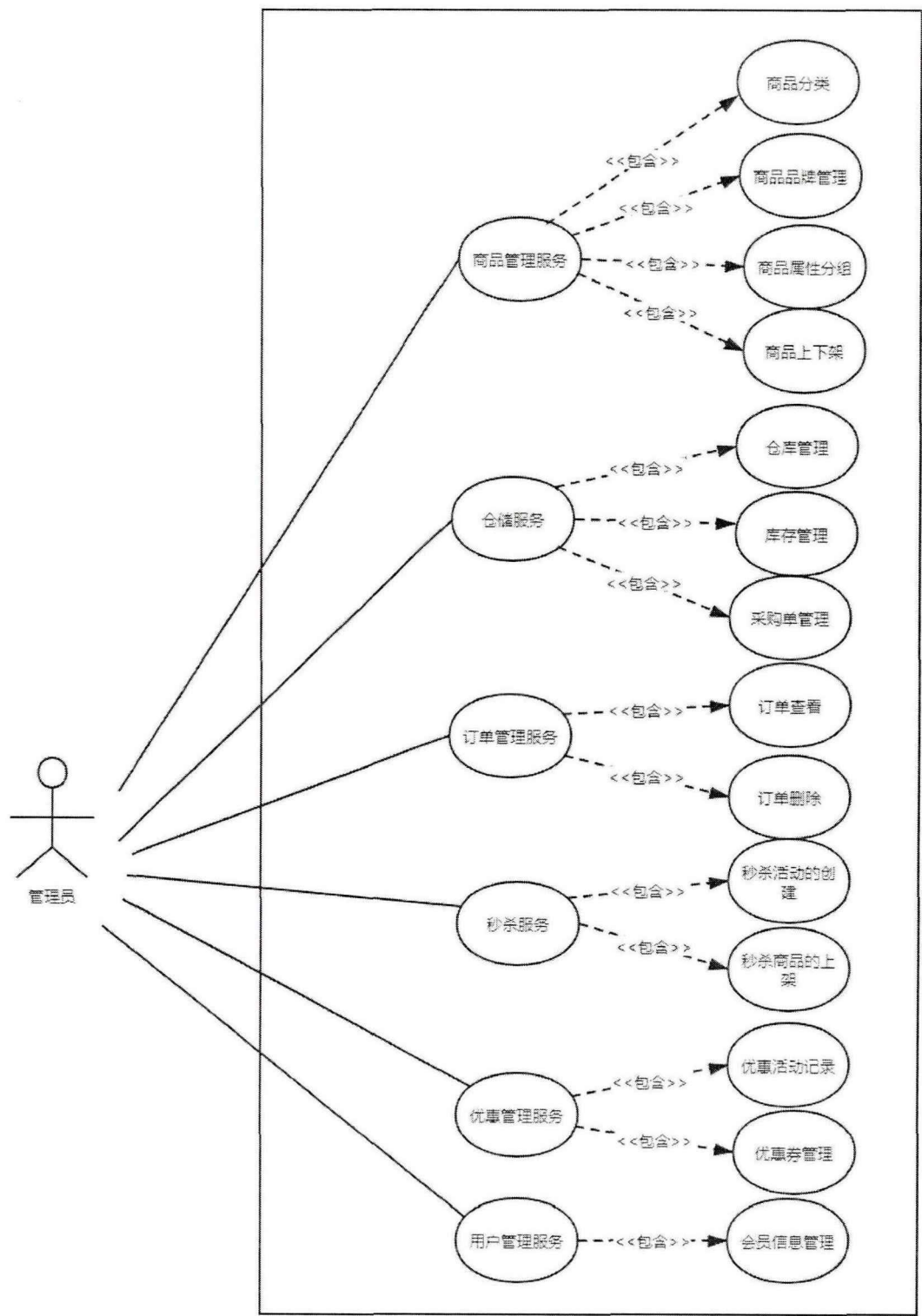


图 3-1 系统管理员用例图

3.3.2 用户的 UML 用例图

用户是系统的主要使用者，系统的功能尽量要满足用户购物的需要。对于用户管理服务来说对于用户管理功能来说主要包括基本信息管理、收货信息管理和个人订单管理，通过可以完善与查看自己的相关信息，不仅可以让他人通过更多

的信息了解自己,同时还可以通过完善的信息提高自己的购物效率,因为只有收货信息完整才能正常的下单;个人订单管理可以保证用户查看自己的订单。购物车管理服务提高了用户的购物体验,用户可以对商品在同一个页面中就可以进行多维度的对比,无需跳转的前提下保证了用户选择的最优,同时只需要简单的点击操作就可以调整货物的数量,极大的提高的下单的成功率,之后还可以将自己不满意的商品进行清除;用户的中央认证服务主要是对用户的登录、注册等进行验证以保证用户能正常的使用系统。用户登录使用单点登录的方式来保证用户一次登录就可使用系统所有功能的能力。同时用户还可以使用社交帐号进行登录,当系统检测到该社交帐号是第一次登录时就会提醒用户进行注册,用户使用自己的手机号配合验证码完成自己的注册信息后将社交帐号与个人账号信息相绑定,方便用户下次社交登录;对于支付服务来说,主要涉及用支付宝的接口来完善用户的支付流程。用户下单付款时先保证用户登录支付宝并记录当前的支付宝信息,然后通过查询支付宝接口的返回信息来判断用户付款,当用户想要发起退款操作时,通过记录的支付宝账号来完成退款的操作。搜索服务是用来方便用户对商品进行查找,商品的检索方式主要有三个入口,包括选择分类、关键词和筛选条件选择。当用户输入关键词时,通过对用户输入的关键词进行分词操作,根据匹配度来返回用户想要查询的商品信息。秒杀服务使用户在秒杀活动中进行秒杀下单操作,抢购自己所需要的商品。订单管理服务中用户使用了立即下单和订单退款功能,保证用户能够正常的购买商品以及当商品不满意时进行退款的操作。系统用户的 UML 用例图如 3-2 所示。

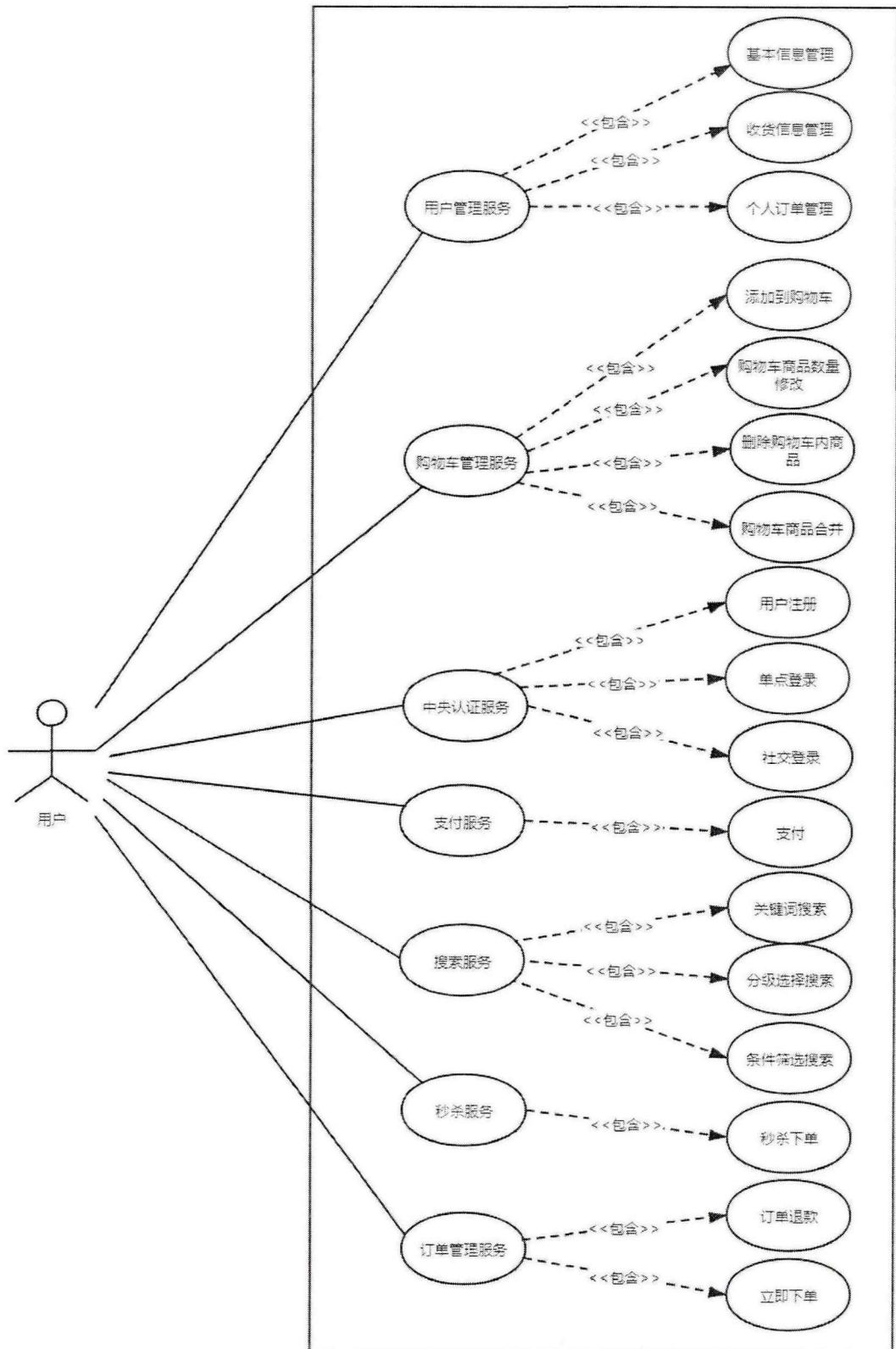


图 3-2 系统用户用例图

3.4 系统的非功能需求分析

对于网上商城系统来说,除了系统的正常运行,满足用户的基本的购物功能外,系统的性能也很重要。接下来从几个方面描述系统的非功能需求。

系统的响应时间:对于网上商城来说,用户的每次点击所等待的时间都将决定用户是否提前取消下单,因此系统的响应时间越短越好。因此对于一些简单的点击操作,系统的响应时间应该在 500 毫秒以下,而对于平均的响应请求,系统的响应时间在 2 秒内,最大的响应时间应该在 6 秒左右。

系统的并发数:系统并发数是指现实系统中操作业务的用户。对于该系统来说,因为有秒杀活动的存在,所以对于系统的并发数有比较高的要求。该系统的用户并发数要满足单秒内 1000 以上的并发量。

系统的安全性:对于网上商城来说,用户的每一次访问都是对系统资源的访问,对于一些特定的接口,要通过鉴权之后才能访问,用户只有通过短信验证码才可以注册。对于用户的个人信息来说,密码是最重要的。因此使用 MD5 (Message-Digest Algorithm 5, 信息摘要算法) 加盐的方式来对密码进行加密处理以防止泄露。

系统的简单易用性:对于网上商城来说,操作简单、界面友好会直接影响用户的购物体验。因此该系统使用单点登录的方式,用户只需登录一次就可以访问该系统的大部分功能。系统的引导性良好,提示明确,能保证用户在初次体验就能够轻松的使用。

3.5 本章小结

该商城系统通过使用角色的不同形式对该系统的需求进行分析,保证系统的功能满足角色的需要。系统功能包括用户管理服务、购物车管理服务、中央认证服务、支付服务、搜索服务、秒杀服务、订单管理服务、商品管理服务、仓储服务和优惠管理服务。商品管理服务是对商品进行管理,包括商品分类、商品品牌管理、商品属性分组、商品上下架等;仓储服务主要对仓库的信息和商品库存信息进行管理;支付服务主要用于用户支付的操作;秒杀服务主要用来帮助用户以很低的价格购买到超值的物品;搜索服务是帮助用户更好的找到自己想要的商品;中央认证服务主要用来用户登录注册时的校验操作;购物车管理服务主要用于一次性支付;订单管理服务主要用来处理订单流转以及订单的状态;用户管理服务主要是用来帮助用户管理自己的信息和管理员管理用户信息。优惠管理服务主要记录优惠活动的记录以及相关的优惠券信息。然后通过用例图,将不同角色

所使用功能的不同通过图表的形式进行一一对应,能够更好的了解角色与功能之间的关系。最后通过系统反映的性能指标几个方面详细介绍了系统所需要的非功能需求。

第四章 系统的总体设计

本章将分别从系统软件层次架构设计、系统功能模块设计、系统的数据库设计、系统的界面设计以及系统的接口上对系统进行总体设计。

4.1 系统软件层次架构设计

电子商务日益成熟，人们对于网购的要求也越来越高，系统对于单秒内将要处理的订单也越来越多，单一应用架构中所有的功能在同一个系统内实现的架构远远达不到需求，服务器在如此大的访问量的作用下将轻易崩溃。为了更好的处理用户的下单请求，使系统能承载更好的并发量，系统采用微服务的架构去设计^[14]。系统软件的层次架构设计如图 4-1 所示。

从系统的表现层来说，系统的使用者包括用户与管理员两种角色。使用者的不同导致前台页面的展示效果以及功能的不同。使用者发起请求在到达网关前经过 Nginx 集群^[15]。网关根据请求动态路由到指定的服务^[16]，如仓储服务、中央认证服务、优惠管理服务等。还有一些第三方服务包括短信验证码、支付接口和第三方登录。同时在服务的运行过程中保持对服务的治理，保证了各服务的可用性，包括注册中心、服务容错、远程调用、服务追踪和分布式事务等。服务首先在注册中心注册，才会被其他服务发现。发起的请求会通过负载均衡使单个服务不会承受过多的请求，但是如果用户发起的请求太多，导致服务无法处理时，这时候就会对服务进行容错处理，包括熔断或者降级，这里使用了 Sentinel。同时网关还要进行对请求进行校验，因为某些服务需要登录才可以访问。而对于一些服务来说，如秒杀服务，要使用缓存的数据来减少对数据库的访问压力，因此要使用 Redis^[17]来保存缓存数据^[18]，减轻 MySQL 的压力。服务之间的消息传递采用 RabbitMQ 来做消息队列，保证了消息的异步消费，提供系统的并发能力，同时使用 Seata 解决分布式问题。而对于搜索服务来说，使用 Elasticsearch 来支持全文检索。在系统中使用 Nacos 来作为服务的注册中心，而对于服务配置可以从配置中心获取配置。对于一个用户行为可行会引起服务间的多层调用，而如果要检查在服务调用时候链路中的哪个部分出现问题，就需要链路追踪，找到问题出现的位置进行修复，这里使用 Sleuth 进行链路追踪。

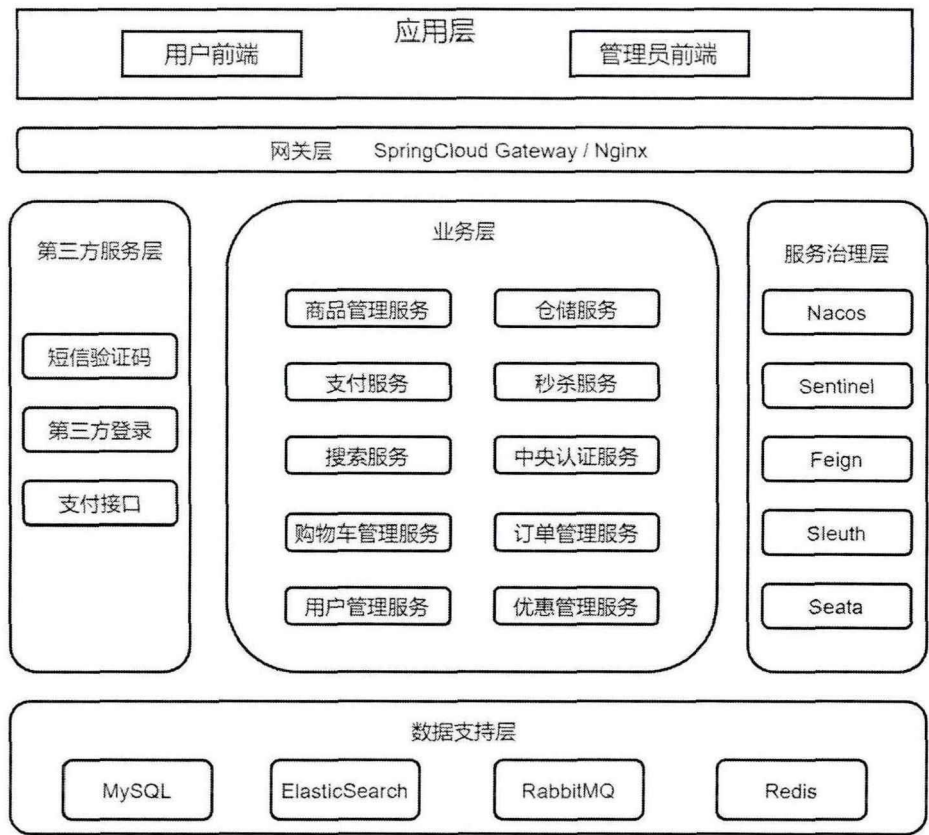


图 4-1 系统层次架构图

4.2 系统微服务架构设计

由于单体架构随着需求迭代会产生种种的问题，因此系统采用微服务的架构进行设计。将系统的功能以单独服务的形式运行，保证了系统的充分解耦，同时使用微服务的组件对相关的服务进行管理，保证服务的可用性。系统服务包括商品管理服务、仓储服务、支付服务、秒杀服务、搜索服务、中央认证服务、购物车管理服务、订单管理服务、用户管理服务和优惠管理服务。用户发起的请求会在网关处理完成后，将请求发送到相应的服务获取相应的功能。而这些服务都是注册在 Nacos 注册中心中，只有在注册中心中注册过的服务才会被其他服务所发现，同时被其他组件所治理。服务的相关配置同样在 Nacos 中进行配置，减少了系统代码中配置修改，服务之间的远程调用通过 Feign 进行调用。由于服务之间部署在不同的机器，因此数据传输的一致性保证就显得尤为重要，这里使用了 Seata 进行分布式管理，保证了数据的一致性。同时系统使用 Sentinel 进行流量控制与服务的熔断降级，避免了单体的服务宕机或者流量过大导致其他服务的不可用，由于系统的调用层级深入，数据传输层层转发，当系统调用失败时，通

过 Sleuth 进行链路的追踪，查询当前链路的失败节点，便于问题的查找与维护。
系统的微服务架构设计图如图 4-2 所示。

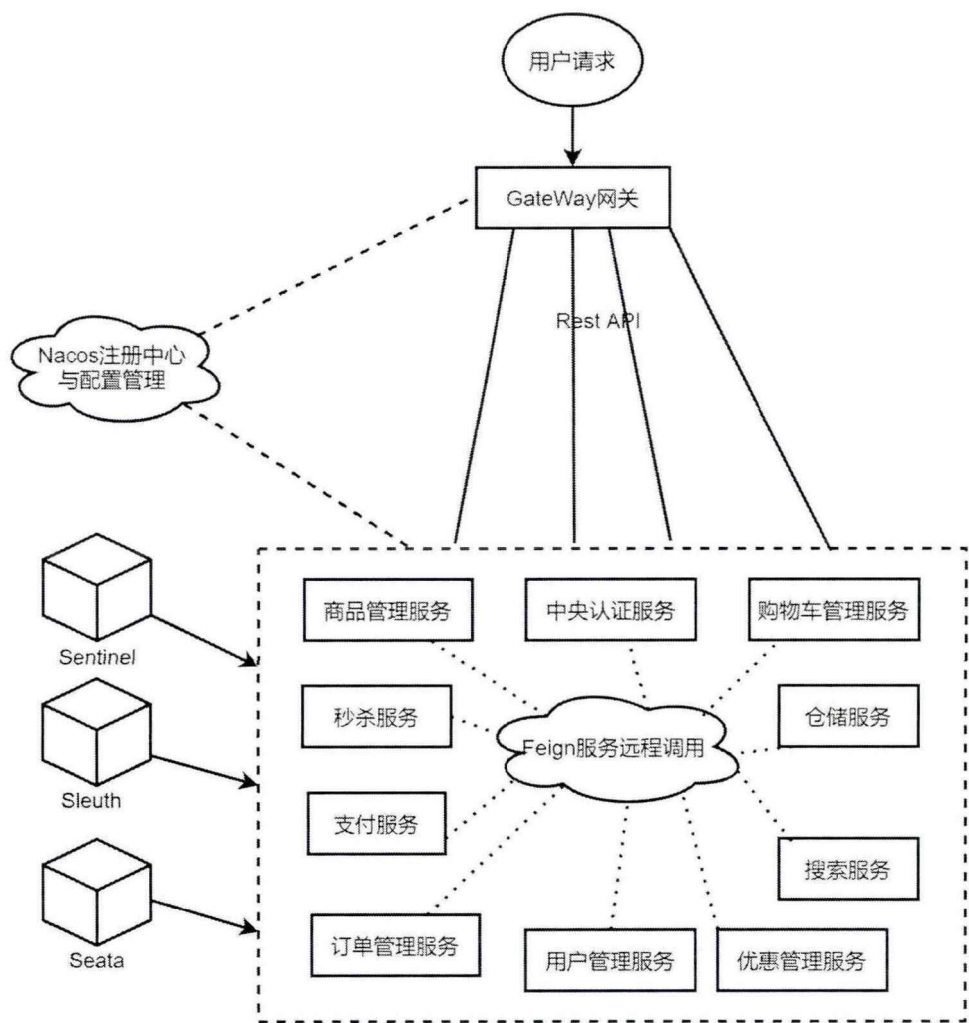


图 4-2 微服务架构设计图

4.3 系统功能模块设计



图 4-3 系统功能模块图

该商城系统主要包括服务包括中央认证服务、商品管理服务、仓储服务、订单管理服务、支付服务、秒杀服务、搜索服务、购物车管理服务、用户管理服务、优惠管理服务。商品管理服务主要提供商品的管理和商品信息维护功能,包括商品分类、商品品牌管理、商品属性分组、商品上下架。仓储服务主要维护商品库存,包括仓库管理、库存管理、采购单管理。支付服务提供支付功能。秒杀服务提供秒杀活动的创建、秒杀商品的上架、秒杀下单。搜索服务提供搜索功能,包括关键词搜索、分级选择搜索和条件筛选搜索。中央认证主要提供验证信息的功能,包括单点登录和社交登录、用户注册。购物车服务提供添加到购物车、购物车商品数量修改、删除购物车内的商品、购物车商品合并。订单管理服务主要包括立即下单、订单查看、订单删除、订单退款。用户服务主要提供用户管理自己信息的一些功能包括基本信息管理、收货信息管理、个人订单管理和会员信息管理。优惠服务包括优惠活动记录,优惠券管理。系统的功能模块图如图 4-3 所示。

4.4 系统的数据库设计

数据库是用来保存系统数据,使数据有组织的进行统一管理。数据库中的数据作为请求响应的底层,数据查询时间的长短将大大影响着系统的响应时间。因此数据库的设计要合理、简洁,保证数据库中数据的查询时间尽量的短。后台数据使用 Mybatis 框架进行设计^[19],接下来系统主要从数据库的 E-R 图和数据库表的设计两方面对系统的数据库进行设计。

4.4.1 E-R 图设计

网上商城以展示商品给用户,用户点击下单为主线进行设计。商品实体是由商品细节组成,商品细节包括商品的价格、商品标题、商品 id、spuId、skuId、默认的图片等属性,商品是展示给用户观看,一个用户可以查看多个商品。用户的属性包括用户的 id、手机号码等,同时一个用户还可以有多个收货信息,收货信息包含收货的地址 id、收件人信息、是否为默认地址等属性,收货信息包含在订单实体中,是由订单创建,而订单是由用户进行创建,一个用户可以创建多个订单。订单的属性包括订单的编号、使用优惠券的 id、创建的时间、用户名、订单金额、订单地址 id、订单状态、支付方式等。一个订单中可能拥有多个商品,一个订单只有一种的支付的方式。对于支付的实体来说包括支付金额、订单 id、支付流水号、支付状态、创建的时间、回调的内容。一个订单中可能拥有多个优惠的活动,优惠的属性包括优惠券 id 和活动 id,其他属性是由秒杀活动组成或者与优惠券关联,一个优惠活动可以有多个优惠券和多个秒杀活动。优惠券的属性包含金额、优惠券 id、优惠券名称、优惠券类型、总数量、单人限制数量、开

始时间、结束时间、已获取数量、发布状态等，秒杀活动的属性包括秒杀 id、活动 id、商品 id、商品名称、秒杀价格、限购数量、开始时间、结束时间、总体数量等。对于商品来说，同样拥有品牌和分类类型，一个商品只有一种品牌，一个品牌可以有多件商品。品牌的属性有品牌 id、品牌名、品牌 logo、品牌介绍，分类类型的属性有层级、分类名称、分类 id。库存的属性有库存 id、skuid、仓库 id、锁定库存、库存数。库存是保存在仓库中，仓库的属性包括仓库 id、仓库地址、仓库名、区域编码。仓库中的商品都是通过采购的渠道进入库存，采购的属性包括采购单 id、采购金额、采购商品 id、采购数量、是否入库等，采购与仓库的关系是多对多的关系。商城系统的 E-R 图如图 4-4 所示。

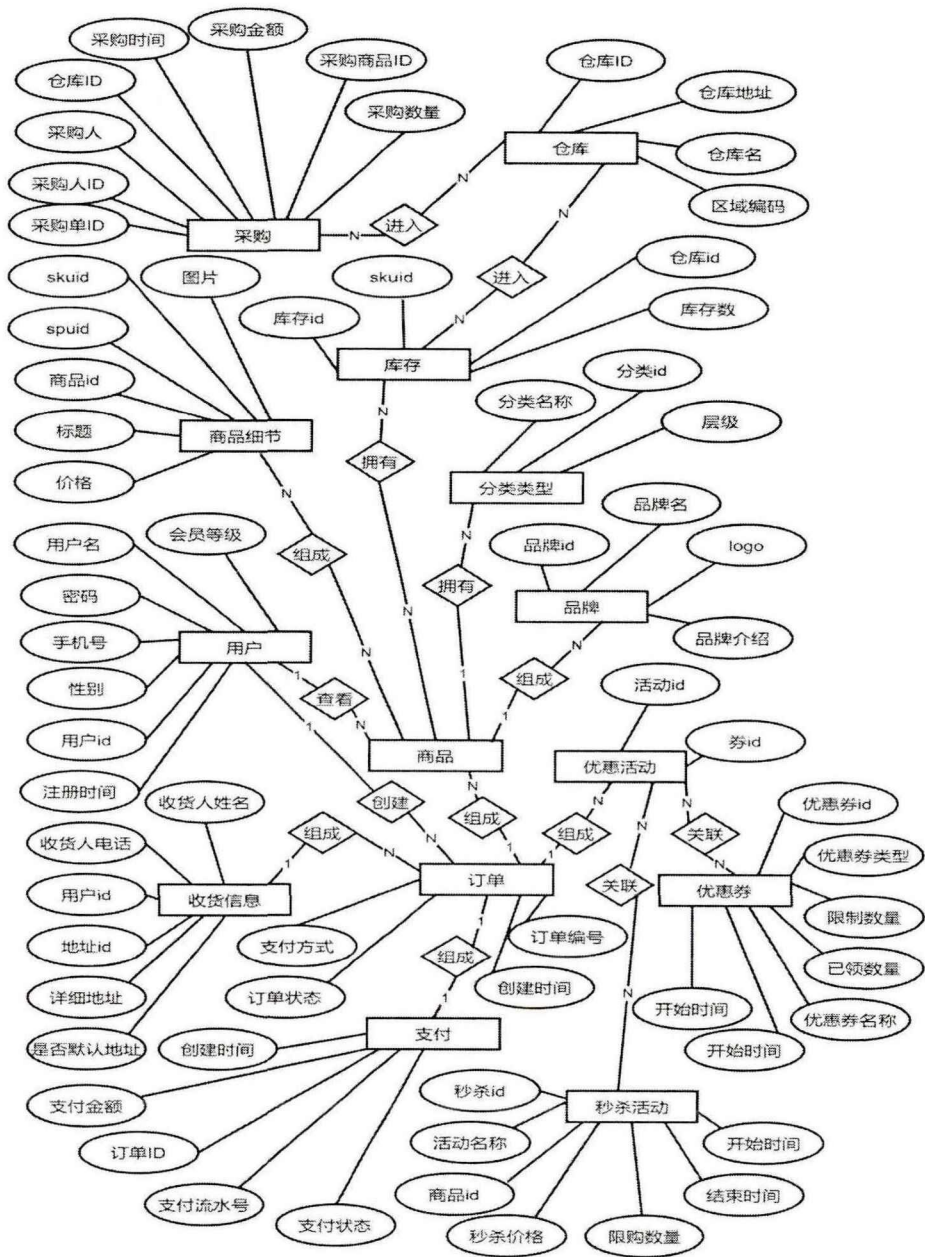


图 4-4 商城系统数据库 E-R 图

4.4.2 数据库表设计

网上商城系统涉及的相关功能有很多,因此数据库表也有很多,表的设计主要由字段名、数据类所示型、长度、是否为空、主键、外键、注释构成。字段名是字段含义的英文翻译,保证见名知意;数据类型是根据该字段所表示的含义进行特定的规定,如数字类型的 `int`、文字类型的 `varchar`、时间类型的 `datetime`;长度的设定用来预估当前字段具体信息的最大长度;是否允许为空要对用户进行限定,对于一些重要的字段,是不能为空的,因为这些字段可能会用来功能上的操作;同时要标明表的主键,一张表只有一个主键,保证能通过主键查找到表的所有信息;外键的表明同样重要,这里因为采用微服务的架构,数据库表的存储位置可能不在同一台机器上,所以对所有的字段都不设置外键,减少连表的操作。

用户表主要描述了用户的相关信息,如表 4-1 所示。字段 `id` 使用户 `id`,用户的 `id` 是整个表的主键,通过 `id` 可以查看用户的所有属性信息,同时 `id` 要设为自增;字段 `username` 表示用户名;字段 `password` 是密码,采用 `varchar` 类型进行存储是因为密码要进行加密,防止相关的人员通过查看数据库表获取到用户的信息,导致数据泄密;字段 `gender` 是用户的性别,用户的性别可以用来首页商品展示推荐相关商品的功能,也可以用来了解系统在男女使用中的比例;字段 `header` 是表示用户的头像,用户头像也是用户辨别其他人身份的凭证之一;字段 `create_time` 表示用注册时间,注册时间可以用来进行用户查找的筛选;字段 `email` 表示用户的邮箱,用户的邮箱可以用来推送相关的优惠信息,用户可以查看邮件获取到最新的优惠活动,这是系统推送广告的唯一途径;字段 `mobile` 表示拥护的手机号,手机号是唯一的,因此手机号是用户登录系统的账号信息,配合密码一起使用方可登录系统;字段 `level_id` 表示用户的会员等级,用户会员等级与相关的优惠信息进行挂钩,某些优惠活动只有特定的等级才能进入;字段 `birth` 表示用户的生日信息,如果当前日期是用户的生日,方便系统给用户发送特有的生日优惠,提高用户参与用户的热情,体现人文关怀。

表 4-1 用户表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
<code>id</code>	用户 ID	<code>bigint</code>	20	是	否
<code>username</code>	用户名	<code>char</code>	64	否	否
<code>password</code>	密码	<code>varchar</code>	64	否	否
<code>gender</code>	性别	<code>tinyint</code>	2	否	否
<code>header</code>	头像	<code>varchar</code>	255	否	否
<code>email</code>	邮箱	<code>varchar</code>	64	否	否

(续上表)

字段名称	解释说明	数据类型	长度	是否为主键	可为空
mobile	手机号码	varchar	11	否	否
create_time	注册时间	datetime		否	否
level_id	会员等级	bigint	20	否	否
birth	生日	date		否	否

收货地址表主要描述了用户收货地址相关的信息，如表 4-2 所示。字段 id 表示当前收货地址的序号；字段 user_id 表示当前地址中用户的 id，通过 user_id 可以判定当前的地址是属于哪个用户的地址；字段 name 表示收件人的姓名，收件姓名可以作为用户收货与快递员的核对信息；字段 mobile 表示收件人电话，手机人电话可以作为用户获取收货消息的渠道，用户通过电话信息获取货物信息后方可取货，同时电话信息页可以作为用户收货与快递员的核对信息；字段 province 表示省份，省份是选择项，防止用户输入不存在的省份；字段 city 表示用户收货地址的城市，城市同样是选择项，防止用户输入不存在的城市；字段 region 表示用户收货的区，区同样是选择项，防止用户输入不存在的区；postal_code 表示邮政编码，邮政编码是为了和收货区域相对应；字段 address 表示用户收货地址的详细信息，用户可以将地址填写到门牌号，方便快递员送货；字段 default 表示该地址是否是默认地址，设置完默认地址后用户访问的商品信息会定位到该地址。

表 4-2 收货信息表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
id	收货序号	bigint	20	是	否
user_id	用户 ID	bigint	20	否	否
name	收件人姓名	tinyint	2	否	否
mobile	收件人电话	varchar	20	否	否
province	省份/直辖市	varchar	20	否	否
city	城市	varchar	20	否	否
region	区	varchar	20	否	否
postal_code	邮政编码	int	6	否	否
address	详细地址	varchar	255	否	否
default	是否默认	varchar	2	否	否

秒杀活动表主要描述了秒杀活动的相关信息，如表 4-3 所示。字段 `id` 表示当前秒杀活动的 `id`，该字段设置为自增的主键；字段 `title` 表示秒杀活动的标题，一个好的会很吸引用户参与活动；字段 `active_id` 表示秒杀的活动 `id`，活动有很多，并且是之前创建的，将本次秒杀与之前的活动关联就可成为新的秒杀活动；字段 `coupon_id` 表示优惠券的 `id`，秒杀或需要和优惠券相关联才能对商品信息进行优惠降价；字段 `goods_id` 表示商品 `id`，代表本次秒杀的商品；字段 `goods_name` 代表商品的名称，表示本次活动的商品名字；字段 `kill_price` 代表本次的秒杀价格，秒杀价格是根据商品的实际价格减去相关联的优惠券之后的最终价格；字段 `start_time` 代表活动的开始时间，时间还未到秒杀时间时会进入秒杀倒计时；字段 `end_time` 代表活动的结束时间，如果当前时间大于活动的结束时间，用户就无法参与本次活动；字段 `all_number` 代表本次活动商品的全部数量，当商品数量为 0 时，本次活动亦结束；字段 `limit_number` 代表本次活动同一个用户所能购买的最大商品数；字段 `notice_type` 表示当前活动的通知方式（1：手机，2：邮箱），用户可以查看通知获取活动的相关信息；字段 `send_time` 表示 0 活动推送的发送时间；字段 `status` 表示活动是否开启（1：开启，0：未开启）；

表 4-3 秒杀活动表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
<code>id</code>	秒杀 <code>id</code>	<code>bigint</code>	20	是	否
<code>title</code>	秒杀标题	<code>varchar</code>	255	否	否
<code>active_id</code>	活动 <code>id</code>	<code>bigint</code>	20	否	否
<code>coupon_id</code>	优惠券 <code>id</code>	<code>bigint</code>	20	否	否
<code>goods_id</code>	商品 <code>id</code>	<code>bigint</code>	20	否	否
<code>goods_name</code>	商品名称	<code>varchar</code>	255	否	否
<code>kill_price</code>	秒杀价格	<code>bigint</code>	20	否	否
<code>start_time</code>	开始时间	<code>datetime</code>		否	否
<code>end_time</code>	结束时间	<code>datetime</code>		否	否
<code>all_number</code>	所有数量	<code>int</code>	20	否	否
<code>limit_number</code>	限制数量	<code>int</code>	20	否	否
<code>notice_type</code>	通知方式	<code>tinyint</code>	2	否	否
<code>send_time</code>	发送时间	<code>datetime</code>		否	否
<code>status</code>	启动状态	<code>tinyint</code>	2	否	否

字段 `id` 表示订单的序号，设为主键；字段 `member_id` 表示用户 `id`，通过用户 `id`，表示当前订单是由哪个用户进行创建；字段 `order_sn` 表示订单号，订单号是在用户创建订单的时候根据时间生成，订单号也可以用来查询订单；字段 `rec_post` 表示收货地址的邮政编码；字段 `coupon_id` 表示优惠券的 `id`，当前订单使用的优惠信息通过优惠券的 `id` 可以进行查找；字段 `create_time` 表示订单创建的时间，订单需要可以用来查看一段时间内订单的数量；字段 `username` 表示订单的用户名；字段 `total_amoun` 表示当前订单的总金额，总金额需要给用户展示方便用户核对，该字段的类型设计 `decimal` 类型，因为数字会有小数点，`decimal` 类型会防止数字溢出。字段 `promotion_amount` 表示订单的优惠金额，订单的优惠价格与优惠券相关的信息绑定，用户可以查看优惠券的优惠信息与订单的优惠信息进行核对；字段 `pay_amount` 表示订单的最终金额，订单的最终金额是由订单的总价格减去优惠价格产生。字段 `status` 表示订单所处的当前位置，订单的所处位置也是由数字进行标识（0：待付款，1：待发货，2：已发货，3：已完成，4：无效订单），管理员可以通过状态信息进行订单的过滤查找；订单总额和订单最终都是使用 `decimal` 类型进行设计。字段 `pay_type` 表示订单的支付类型由数字进行标识；字段 `rec_name` 表示收件人；字段 `rec_tel` 表示收件电话；字段 `note` 表示备注，包含用户对订单的特殊要求；字段 `payment_time` 表示付款的时间；字段 `rec_address` 表示收货的详细地址；字段 `rec_time` 表示订单的收货时间。订单表如表 4-4 所示。

表 4-4 订单表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
<code>id</code>	订单序号	<code>bigint</code>	20	是	否
<code>member_id</code>	用户 ID	<code>bigint</code>	20	否	否
<code>order_sn</code>	订单号	<code>bigint</code>	20	否	否
<code>coupon_id</code>	优惠券 id	<code>bigint</code>	20	否	否
<code>create_time</code>	创建时间	<code>datetime</code>	20	否	否
<code>username</code>	用户名	<code>varchar</code>	20	否	否
<code>total_amount</code>	订单总额	<code>decimal</code>	(18, 2)	否	否
<code>pay_amount</code>	支付款数	<code>decimal</code>	(18, 2)	否	否
<code>promotion_amount</code>	优惠金额	<code>decimal</code>	(18, 2)	否	否
<code>pay_type</code>	支付类型	<code>varchar</code>	2	否	否
<code>status</code>	订单状态	<code>tinyint</code>	2	否	否
<code>rec_name</code>	收件人	<code>varchar</code>	32	否	否

(续上表)

字段名称	解释说明	数据类型	长度	是否为主键	可为空
rec_tel	收件号码	varchar	32	否	否
rec_post	收件邮编	varchar	32	否	否
rec_address	收件住址	varchar	32	否	否
note	订单备注	varchar	32	否	否
payment_time	支付时间	datetime		否	否
rec_time	收货时间	datetime		否	否

支付表主要描述支付相关的信息,如表 4-5 所示。字段 id 表示支付的序号,支付序号设为主键;字段 order_id 表示订单 id,可以通过订单 id 查询相关的订单信息;字段 ali_no 表示支付宝订单号,支付宝订单号是在用户支付宝支付时根据时间生成;字段 all 表示支付的总金额;字段 location 表示支付当前所处的位置(0:未付款;1:已付款);字段 create_time 表示当前的支付是什么时候被建立;字段 callback_content 表示支付回调的内容,回调内容一般为用户的个人信息,以便用户退款时使用。

表 4-5 支付表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
id	支付序号	bigint	20	是	否
order_id	订单 id	bigint	20	否	否
ali_no	支付宝订单号	varchar	30	否	否
all	付款钱数	decimal	(18, 2)	否	否
location	支付所处位置	tinyint	2	否	否
create_time	创建时间	datetime		否	否
callback_content	回调内容	varchar	100	否	否

商品表主要描述商品相关的信息,如表 4-6 所示。字段 id 表示商品 id,代表商品的编号,商品 id 设为主键;字段 spu_id 表示商品 spu_id,可以通过商品 spu_id 可以查询 spu 的所有信息;字段 sku_id 表示商品 sku_id,可以通过商品 sku_id 可以查询 spu 的所有信息;字段 title 表示商品名称;字段 first 表示商品的首字母;字段 price 表示售价。

表 4-6 商品表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
id	商品 id	bigint	20	是	否
spu_id	商品 spu_id	bigint	20	否	否
sku_id	商品 sku_id	bigint	20	否	否
title	商品名称	varchar	100	否	否
first	商品首字母	varchar	2	否	否
url	商品图片	varchar	100	否	否
price	价格	decimal	(18, 2)	否	否

商品细节表主要描述商品详情相关的信息，如表 4-7 所示。字段 spu_id 表示商品 spu_id，商品 spu_id 设为主键；字段 create_time 表示当前商品是什么时候被建立；字段 spu_desc 表示商品的描述信息，描述信息是对商品的解释说明，通过商品描述可以更好的了解商品情况；字段 catalog_id 表示商品分类 id；字段 publish_status 表示当前商品的上架状态(0: 为未上架; 1: 上架); 字段 update_time 表示修改时间。

表 4-7 商品细节表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
spu_id	商品 spu_id	bigint	20	是	否
create_time	创建时间	datetime		否	否
spu_desc	商品描述	varchar	200	否	否
catalog_id	分类 id	bigint	20	否	否
brand_id	品牌 id	bigint	20	否	否
publish_status	当前商品状况	tinyint	2	否	否
update_time	修改时间	datetime		否	否

库存表主要描述商品库存相关的信息，如表 4-8 所示。字段 id 表示库存的序号，库存序号设为主键；字段 sku_id 表示商品的 id，可以通过商品 id 获取商品详细信息；字段 ware_id 表示仓库 id，代表当前商品所在的仓库；字段 stock 表示商品库存数，当当前库存数量不足时就要对商品进行下架同时进行采购业务；字段 sku_name 表示当前库存商品的名称；字段 stock_locked 表示库存中被锁定商品的数量。

表 4-8 库存表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
id	库存序号	bigint	20	是	否
sku_id	商品 id	bigint	20	否	否
ware_id	仓库 id	bigint	20	否	否
stock	库存数	int		否	否
sku_name	商品名称	varchar	20	否	否
stock_locked	被锁定库存	int		否	否

采购表主要描述商品采购相关的信息，如表 4-9 所示。字段 id 表示采购单号，采购单号设为自增的主键；字段 ass_id 表示采购人 id；字段 ass_name 表示采购人姓名；字段 status 表示采购状态（0：未采购；1：已采购；2：已入库）；字段 ware_id 表示仓库 id，代表当前采购的商品将保存到哪个仓库；字段 sku_id 表示采购商品 id；字段 sku_num 表示采购数量；字段 amount 表示采购金额；字段 create_time 表示采购时间；字段 amount 表示采购金额；字段 update_time 表示采购更新时间。字段 tel 表示采购人电话，可以通过以上采购人信息对采购者进行记录，保证采购有迹可循。

表 4-9 采购表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
id	采购单号	bigint	20	是	否
ass_id	采购人 id	bigint	20	否	否
ass_name	采购者	varchar	20	否	否
status	采购状态	tinyint	2	否	否
ware_id	仓库 id	bigint	20	否	否
sku_id	采购商品 id	bigint	20	否	否
sku_num	采购数量	int	20	否	否
tel	采购人电话	char	11	否	否
amount	采购金额	decimal	(18, 2)	否	否
create_time	采购时间	datetime		否	否
update_time	更新时间	datetime		否	否

仓库表主要描述仓库相关的信息，如表 4-10 所示。字段 id 表示仓库 id；字段 name 表示仓库名；字段 address 表示仓库地址；字段 areacode 表示采购时间；字段 amount 表示仓库的区域编码。

表 4-10 仓库表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
id	仓库 id	bigint	20	是	否
name	仓库名	varchar	255	否	否
address	仓库地址	varchar	255	否	否
areacode	区域编码	varchar	20	否	否

分类类型表主要描述商品分类相关的信息，如表 4-11 所示。字段 cat_id 表示分类 id；字段 name 表示分类名称；字段 parent_cid 表示父分类 id；字段 cat_level 表示层级；字段 product_count 表示商品数量。

表 4-11 分类类型表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
cat_id	分类 id	bigint	20	是	否
name	分类名称	char	50	否	否
parent_cid	父分类 id	bigint	20	否	否
cat_level	层级	int	10	否	否
product_count	商品数量	int	10	否	否

品牌表主要描述商品品牌相关的信息，如表 4-12 所示。字段 brand_id 表示品牌 id；字段 logo 表示 logo 地址；字段 name 表示品牌名称；字段 descript 表示品牌介绍；字段 show_status 表示品牌显示状态（0：显示；1：未显示）。

表 4-12 品牌表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
brand_id	品牌 id	bigint	20	是	否
logo	logo 地址	varchar	100	否	否
name	品牌名称	char	50	否	否
descript	品牌介绍	longtext		否	否
show_status	显示状态	tinyint	2	否	否

优惠活动表主要描述优惠活动相关的信息，如表 4-13 所示。字段 id 表示活

动 id; 字段 coupon_id 表示优惠券 id; 字段 name 表示活动名称; 字段 status 表示活动的启动状态 (0: 未启动; 1: 启动)。

表 4-13 优惠活动表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
id	活动 id	bigint	20	是	否
coupon_id	优惠券 id	bigint	20	否	否
name	活动名称	varchar	255	否	否
status	启动状态	tinyint	2	否	否

优惠券表主要描述优惠券相关的信息, 如表 4-14 所示。字段 id 表示优惠券 id; 字段 start_time 表示活动开始时间; 字段 cou_type 表示优惠券类型 (1: 满减优惠券; 2: 限时优惠券; 3: 等级优惠券); 字段 cou_name 表示优惠卷名称; 字段 num 表示优惠券数量; 字段 amount 表示优惠券的金额; 字段 rec_count 表示已领取数量; 字段 per_limit 表示单人限制数量; 字段 end_time 表示活动结束时间; 字段 publish 表示优惠券的发布状态 (0: 未发布; 1: 发布)。

表 4-14 优惠券表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
id	优惠券 id	bigint	20	是	否
start_time	开始时间	datetime		否	否
cou_type	优惠券类型	tinyint	2	否	否
cou_name	券名	varchar	50	否	否
num	数量	int		否	否
amount	金额	decimal	(18, 2)	否	否
rec_count	已领	int	8	否	否
per_limit	单人限制数量	int		否	否
end_time	结束时间	datetime		否	否
publish	发布状态	tinyint	2	否	否

订单退货表主要描述了用户退货退款的相关信息, 如表 4-15 所示。字段 id 表示退款的序号, 该字段设置为自增主键; 字段 sku_id 表示商品的编号, 商品编号是退款的商品 id, 可以通过该 id 查询该商品的所有信息; 字段 order_sn 表示订单的编号, 该订单编号是用户购买该商品的下单编号, 可以通过该编号了解用户下单的所有信息; 字段 create_time 表示退款的申请时间; 字段 coupon_id 表示优惠券的 id, 该 id 做为后续用户收到退款后, 将相应的优惠券补给用户和库存; 字段 return_amount 表示退款的金额, 退款的金额就是用户下单的付款金

额，用户最终收到的退款数目可以和下单数目进行核算；字段 `refund_channe` 表示退款的渠道，退款的渠道是用户下单的渠道相一致（1：表示支付宝，2：表示微信，3：表示银联），数字标识必须一样，即原路返回；字段 `return_name` 表示退款人，退款人的姓名将作为后期的查询信息的凭证；字段 `return_phone` 表示退款人电话，退款人的电话也将作为后期的查询信息的凭证；字段 `return_location` 表示当前退款所处的位置，使用数字进行标识（0：待处理，1：退货中，2：已完成）；字段 `sku_name` 表示退款商品的名称；字段 `sku_img` 表示退款商品的图片；字段 `sku_count` 表示退款商品的数量，退款的数量与用户下单的数量一致；字段 `sku_price` 表示商品的单价，该单价是商品的基础单价；字段 `sku_real_price` 表示商品的最终单价，该单价是用户最终的购买单价；字段 `reason` 表示退款的原因，管理员对原因进行查看后可以了解该商品的不足从而做出相应的调整；字段 `receive_time` 表示商品收货的时间，某些商品可能在收货一段时间内才可以进行退货退款的操作。

表 4-15 订单退货表

字段名称	解释说明	数据类型	长度	是否为主键	可为空
<code>id</code>	退款序号	<code>bigint</code>	20	是	否
<code>sku_id</code>	商品编号	<code>bigint</code>	20	否	否
<code>order_sn</code>	订单编号	<code>bigint</code>	20	否	否
<code>create_time</code>	创建时间	<code>datetime</code>		否	否
<code>coupon_id</code>	优惠券 id	<code>bigint</code>	20	否	否
<code>return_amount</code>	退款总额	<code>decimal</code>	(18, 2)	否	否
<code>refund_channe</code>	退款渠道	<code>tinyint</code>	2	否	否
<code>return_name</code>	退款人	<code>varchar</code>	20	否	否
<code>return_phone</code>	退款电话	<code>varchar</code>	20	否	否
<code>return_location</code>	当前退款所处位置	<code>tinyint</code>	2	否	否
<code>sku_name</code>	商品名称	<code>varchar</code>	32	否	否
<code>sku_img</code>	商品图片	<code>varchar</code>	32	否	否
<code>sku_count</code>	退货数量	<code>int</code>	32	否	否
<code>sku_price</code>	售卖价格	<code>decimal</code>	(18, 2)	否	否
<code>sku_real_price</code>	商品最终价格	<code>decimal</code>	(18, 2)	否	否
<code>reason</code>	退款原因	<code>varchar</code>	255	否	否
<code>receive_time</code>	收货时间	<code>datetime</code>		否	否

4.5 系统的界面设计

商城系统是为了满足用户购物需求，帮助用户能够快速购买到适合自己商品的系统。因此系统界面要设计的简单明了，符合大众审美体验，对于同一个页面，不要展示给用户太多的信息，防止对用户进行干扰，同时可以起到信息的过滤作用，系统的功能上设计要简单易用，并且要能够满足用户基本的购物需求。对于系统的管理员来说，一个整体界面要包含所有的管理功能，方便管理员在不做页面跳转的前提下进行管理，降低管理的成本。接下来将从几个方面对系统界面进行设计。

4.5.1 后台管理系统的界面设计

后台管理系统采用大块平铺的形式展示给管理员，界面简洁大方，各个分区采用不同的颜色来进行区分。左侧功能导航栏暗灰色进行分区，提供管理员需要的相关功能，点击功能栏功能可以进行二级导航，将一级功能进行展开，多级功能可以进行多级展开。界面上层的导航栏显示系统名称，以淡蓝色进行分区，右侧显示用户个人信息的展示。往下是整体的功能页，采用淡灰色的颜色进行区分，并且分为五层。第一层显示当前的访问路径，管理员了解当前的访问路径，以便回溯；第二层显示搜索的一些功能，包括搜索框、筛选条件选择等，而对于点击按钮采用绿色方便用户进行点击；第三层展示当前展示信息的属性名称，方便管理员了解当前展示字段的含义；第四层展示当前具体的数据，同时可以对展示的行数据进行一些操作，操作按钮同样使用绿色；第五层右侧显示当前数据的页码数，总页数和跳转页。界面设计整体简约大方，采用上下左右的分区展示，功能布局搭配合理，界面设计如图 4-5 所示。

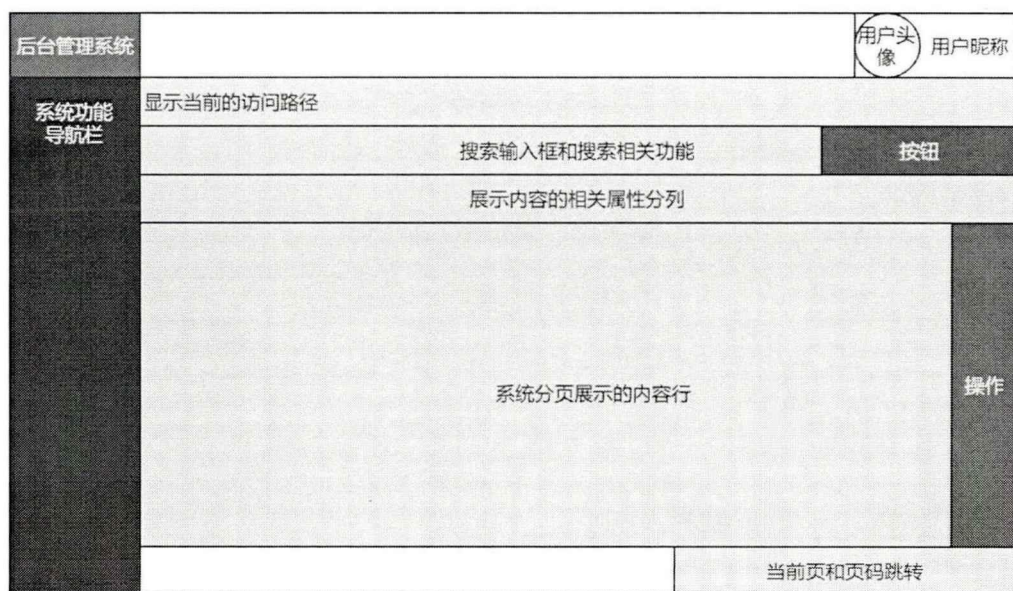


图 4-5 后台管理界面设计图

4.5.2 前台展示的界面设计

前台分户采用同样采用分块模式进行设计,底层颜色使用白色,左边分别是商城 logo 和商品的多级分类,多级分类是用户搜索商品的一种方式,界面右侧采用多层设计,共分为 5 层,第一层为登录注册以及订单查询,这三个按钮都采用淡灰色的颜色,方便用户视觉观测。第二层是搜索框和搜索按钮,作为用户搜索的第二种方式,右侧为购物车,方便用户添加购物车时查看,搜索按钮和购物车都采用红色,突出功能效果,让用户获得视觉冲击。第三层为导航条,以商品的类型、品牌等作为导航关键字,同样采用淡灰色以区分其他分区。第四层为展示的商品轮播图,通过商品轮播让用户了解当前热卖的商品。第五层为固定商品的展示,一般可以作为广告位,展示被广告的商品。再往下整体一块为商城的秒杀活动区,秒杀活动会展示秒杀倒计时,让用户了解活动的结束时间,秒杀活动和秒杀时间都采用红色作为字体颜色,表示秒杀的火爆程度。往下会展示一些秒杀的商品,便于用户抢购。此页面设计围绕“展示”进行设计,更多的展示商品,让用户能第一时间观察到更多的商品,同时布局简单,可复用性强,只需修改部分分区就可用在其他门户类的界面展示,前台展示界面如图 4-6 所示

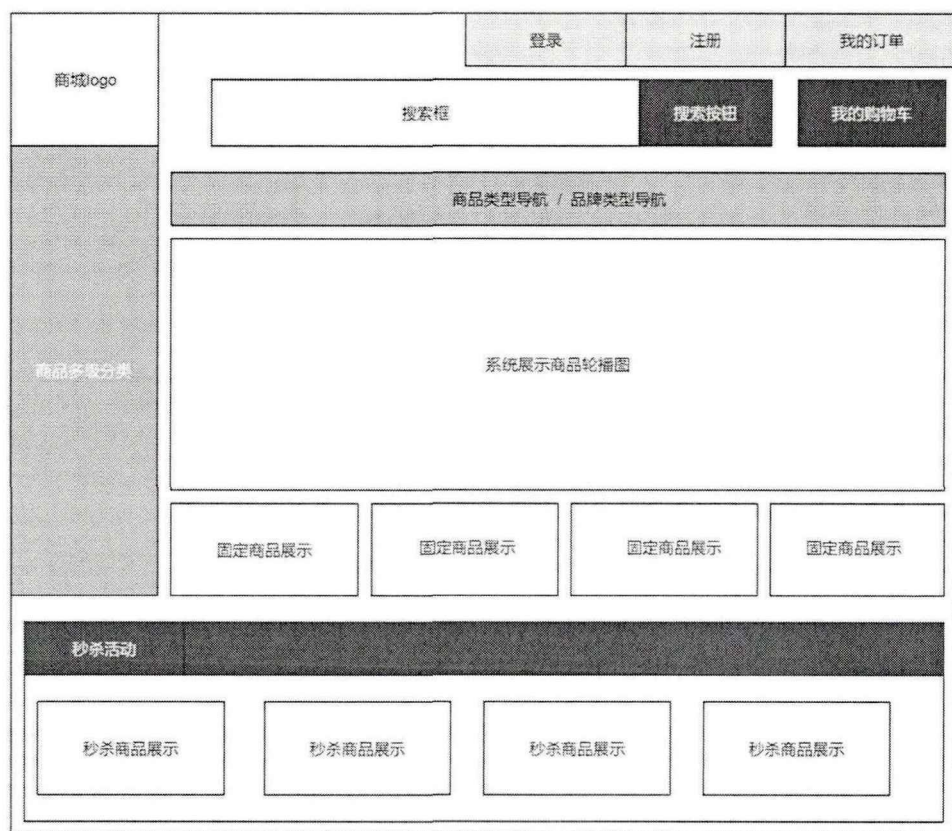


图 4-6 前台展示界面设计图

4.5.3 订单查看的界面设计

对于订单查看来说,要保证订单在能选择不同的订单状态下进行筛选,方便查看各种类型的订单信息。界面第一层为商城 logo,搜索按钮和购物车;第二层

为订单选择条件以及订单搜索框,帮助用户进行订单的筛选;第三层为订单的相关属性以及具体的订单的信息展示,还有对订单的一些操作。此页面只要围绕“查看”进行设计,要对用户的订单展示更多的信息,同时要帮助用户进行订单的搜索,快速找到自己关心的信息。界面如图 4-7 所示。

商城logo	首页	账户设置	搜索框 / 搜索按钮	我的购物车
我的订单				
全部订单	待收货订单 / 待付款订单		订单搜索框 / 搜索按钮	
订单相关的属性和操作,包括订单详情、收件人、金额、取消订单等				
订单时间/订单号 订单属性和操作对应的具体信息				
订单时间/订单号 订单属性和操作对应的具体信息				
订单时间/订单号 订单属性和操作对应的具体信息				
				订单当前展示页/跳转页

图 4-7 订单查看界面设计图

4.5.4 结算页的界面设计

订单结算页的界面设计如图 4-8 所示。界面设计完全采用上下层的形式进行,每一层包括订单结算用户所关心的不同内容。通过分层,帮助用户对信息进行归类,方便用户核对信息。第一层为收货人信息,此层保证用户能够收到购买的货物。第二层为支付方式,用户可以选择自己喜欢的方式进行支付,这里是支付宝支付。第三层为货物的清单,将用户购买的商品集中在一起列出清单,帮助用户核对商品信息。最下层为具体的金额数,要给用户显示购买商品的初始价格、最终价格。这里用户最关心的数目,使用红色标识。钱数往下就是下单按钮,同样用红色标识,便于用户点击。此界面的设计围绕“下单”进行设计,对于用户想要下单所关心的所有信息都要展示,让用户更好的了解自己将要下的订单。

商城logo			我的订单
收件人信息			
支付方式			
货物清单			
优惠情况			
		商品总价格/优惠减少 价格	
		最终付款价格	
		订单提交按钮	

图 4-8 结算页界面设计图

4.5.5 商品搜索页的界面设计

商品搜索页界面设计如图 4-9 所示。界面上半部分完全复用系统门户的界面设计模式，下半部分替换分区的展示，形成商品搜索页。商品搜索一共有三个入口，为搜索框，分类搜索，筛选条件选择。在固定展示区往下会显示当前的访问路径，同时会设计一块分区作为筛选条件的选择区域，筛选的条件包括品牌、价格、大小尺寸等。再往下一块区域为特有商品的筛选条件，如手机电脑的内存筛选等，同时会展示最后包含用户所有筛选条件的最终展示商品。此页面布局完全围绕用户的“筛选”进行设计。用户可以在此界面进行多条件、多级别、多类型的选择，能够帮助用户更好的定位到自己满意的商品。

		登录	注册	我的订单	
商城logo	搜索框和搜索按钮			我的购物车	
全部商品分类	商品类型导航 / 品牌类型导航				
固定商品展示层					
当前访问路径					
品牌类型筛选条件 价格类型筛选条件 大小尺寸筛选条件					
同类型商品展示区	其他筛选条件展示				
	过滤后的商品展示区，显示商品部分详情				
					当前访问页与跳转页

图 4-9 商品搜索页界面设计图

4.5.6 商品详情页的界面设计

系统的商品详情页的界面设计如图 4-10 所示。商品详情页是为了帮助用户更好的了解商品，此界面要让商品的信息尽可能的多，对于商品的介绍尽可能的详细。界面的上半部分同样为复用的基本设计，从访问路径开始，界面将开始介绍商品详情。在页面的左边展示商品的大图，因为系统的商品有很多，系统需要展示的商品有很多，所以在门户页或者搜索页的单个商品展示所占有的位置都不会很大，在商品详情页进行大图展示能更好的帮助用户观察到商品的细节。同时下面要有多角度的展示，用户点击角度的图片同样会以大图展示。商品图片的右侧展示了一些用户相对比较关心的数据，包括商品名称、商品价格、大小颜色等，此时相信用户心里对于商品是否购买已经有了大概的定论，然后在下方分区展示

商品数量的选择、购买或者加入到购物车，这里可以对用户下单起到引导作用。再下方分为左右两侧，左侧为同类型商品展示，不仅可以提高其他产品的曝光率，同时还使用户心理进行比较。右侧为当前的商品所有详尽信息，如果用户还想了解更多的信息，就可以观看此部分的信息，该信息包括商品的 SPU 和 SKU 的所有属性。此界面围绕“详情”进行，要让商品展示的信息全面细致，让用户对商品有全面的了解。



图 4-10 商品详情页界面设计图

4.6 系统接口设计

接口是获取数据的路径，接口设计的准确性影响着对数据获取的难易程度。下面从接口表的设计上，简单介绍一下该系统前端界面获取后端数据的接口以及服务间相互调用获取响应数据的接口设计。这两种接口的设计规则相同，因此在

这里进行统一的描述。系统部分的接口表如表 4-16 所示。

表 4-16 接口表

接口名	请求方法	请求类型	返回类型	含义
/product/category/list	GET	JSON	JSON	获取商品所有分类
/product/attrgroup/info	GET	JSON	JSON	获取属性分组
/product/attr/save	POST	JSON	JSON	保存属性
/product/spu/save	POST	JSON	JSON	新增商品
/member/list	GET	JSON	JSON	获取用户列表
/ware/sku/list	GET	JSON	JSON	查询商品库存
/ware/purchase/received	POST	JSON	JSON	增加采购单
/coupon/coupon/list	GET	JSON	JSON	获取优惠券

接口的设计包括接口名、请求方法、请求类型、返回类型、含义。首先，接口的接口名采用分层设计，接口的第一层代表着接口所在的服务以及功能主体，如 product 代表该接口是在商品服务下，代表了对商品进行操作的功能，member、ware、coupon 分别代表了用户、仓储、优惠等服务的功能，接口的第二层代表该功能下面的属性名称，如 category、attrgroup、purchase、coupon 代表了该服务的类型、属性、采购单、优惠的属性，而对于接口的第三层是对当前属性的一些操作，如 list、save 代表列表获取和修改保存。在接口的请求方法设计中主要是由该方法对数据产生的影响进行设计，如果该方法会修改数据，就将请求方法设计为 POST，如果只是获取数据而不对数据产生影响，就设计为 GET 方式，JSON 进行数据传输，因为 JSON 更易读，同时对于数据的格式转换更简单。在接口的设计中，一定要做到见名知意，同时对接口要进行含义的解释说明，帮助开发人员更好的对接口进行维护，同时避免命名冲突。

4.7 本章小结

本章节以系统的软件层次架构设计开始，对系统进行了总体的设计。接着从系统的功能模块进行设计，详细具体设计了服务所包含的功能，其所有服务包含的功能保证了用户和管理员的基本使用。在数据库层面，从 E-R 图设计描述了各实体之间的关系，从介绍各表中详细的字段的角度出发，主要描述了数据库表的设计准则和规范。在系统的界面设计方面，从后台管理、前台门户、订单查看、结算页、商品搜索页、商品详情页几个关键的界面进行了设计和描述。在系统的接口设计层面，主要通过几个接口描述了相关接口设计的规范。

第五章 系统主要功能模块的详细设计与实现

该商城系统是以微服务的架构进行设计,因此系统的功能都是以服务进行划分,在对微服务的组件进行设计之后,接下来将从中央认证服务、秒杀服务、订单管理服务、支付服务、商品管理服务的功能模块进行具体详细设计与实现。

5.1 微服务基础组件的详细设计与实现

系统的功能模块是以微服务的形式独立运行,因此单体服务的可用性决定了系统的整体可用性,服务的治理在微服务的架构中显得尤为重要,治理的能力决定了服务的可用性。而对于服务的治理,包括服务的注册与发现、服务的远程调用、熔断降级、网关的使用、链路追踪等。

5.1.1 服务的注册与发现的设计与实现

以微服务的形式运行的功能需要在注册中心进行注册才会被其他的服务所发现,将每一个服务设置一个单独的端口作为其他服务访问的路劲,这里采用了Nacos 作为系统的注册中心。在配置中将服务的端口注册到 Nacos 中,Nacos 通过自己的可视化界面管理着这些服务的运行状态与端口号。服务的注册就是将自己的端口更新到 Nacos 的服务注册表中,这些注册表中的服务可以被其他可信服务访问。服务通过遍历注册表中存活的服务端口,对相应的功能进行访问。Nacos 作为服务的注册与发现中心,实现的步骤可以包括修改 pom 文件、配置 Nacos Server 的地址和开启服务的注册与发现。

1. 修改 pom 文件,引入 Nacos 的依赖

```
<dependency>
    <groupId>com.alibaba.cloud</groupId>
    <artifactId>spring-cloud-starter-alibaba-nacos-discovery</artifactId>
</dependency>
```

2. 在配置文件中配置相关的地址

```
spring.cloud.nacos.discovery.server-addr=127.0.0.1:8848
```

3. 在启动类上加入@EnableDiscoveryClient 注解,开启服务的注册与发现功能。而对于独立运行的服务需要设置自己的服务名与端口名才会被注册上去。

```
spring.application.name=service-provider
server.port=8000
```

5. 1. 2 服务的远程调用的设计与实现

服务在注册中心中进行注册，而服务由于不在同台机器上运行，因此服务的远程调用就显得尤为重要。通过使用 Feign 的 HTTP 请求模板，只需要编写简单的接口和插入注解即可。Feign 的工作原理如图 5-1 所示。在获取到调用请求后，通过动态代理机制，基于 requestBean 动态生成 request，之后 encoder 将当前访问的 beanbao 包装成请求发送给拦截器，并在日志中进行记录，在重试器下发送 Http 请求，获取相应的服务，并在获取到服务后将服务的调用原路返回。

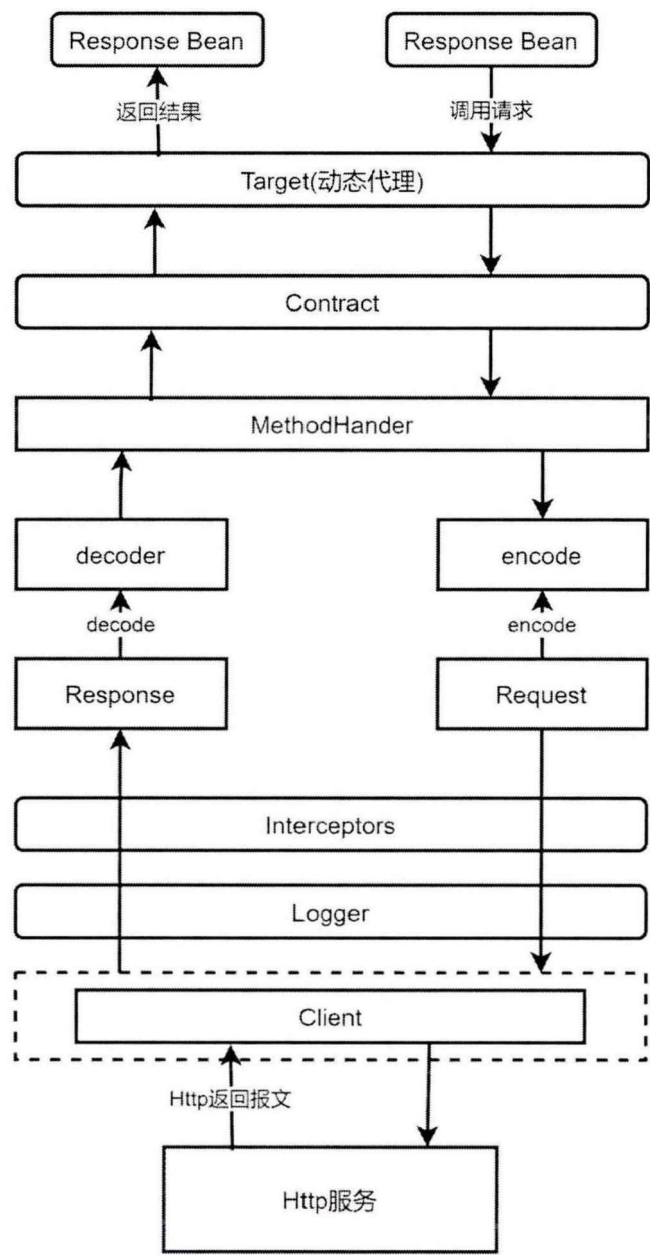


图 5-1 Feign 的工作原理

对于 Feign 的使用，具体的操作步骤如下：

1.引入 pom 依赖


```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

2.使用@ EnableFeignClients 注解开启 feign 功能

```
@EnableFeignClients(basePackages = "com.mall.pms.feign")
```

3.使用@FeignClient 注解声明远程接口

```
@FeignClient("mall-ware")
```

5.1.3 服务的熔断降级的设计与实现

当流量处于高峰期时，服务往往由于突增的流量导致服务的宕机，而对于一些服务，在流量突增时进行有策略的降级和熔断是有必要的。防止单点的服务宕机导致整个系统的不可用。这里采用了 Sentinel 组件对系统进行流量的控制，保证系统的高可用性。可以通过 Sentinel 的控制台查看数据的实时监控，通过观察实时监控，可以设置良好的熔断降级策略。而对于 Sentinel 的使用，具体的操作步骤如下：

1.首先修改配置文件，引入 pom 依赖

```
<dependency>
  <groupId>com.alibaba.cloud</groupId>
  <artifactId>spring-cloud-starter-alibaba-sentinel</artifactId>
</dependency>
```

2.编写 yml 文件

```
spring:
  application:
    name: sentinel-service-3
  cloud:
    sentinel:
      transport:
        dashboard: localhost:8080 #sentineldashboard 地址
        port: 8719 #如果被占用会增加 1
```

3.在 RestController 接口上加入@SentinelResource 即可

5.1.4 服务的网关的设计与实现

网关做为服务的入口，需要将用户的请求通过路由转发的方式，在过滤了非法请求之后，限制入口流量，保证服务接收到无害的用户请求。同时不同服务间调用存在的跨域问题、服务多次调用的鉴权问题都可以通过网关进行管理。这里

使用了 Gateway 组件做为系统的网关层。Gateway 能够提供更简单有效的方式对 API 的访问进行路由转发。Gateway 的工作原理如图 5-2 所示。客户端发送请求给网关，HandlerMapping 在判断请求满足路由后就将请求发送给 WebHandler，之后通过一系列的过滤后，请求就会到达 Service，之后 Service 处理之后原路返回给客户端。

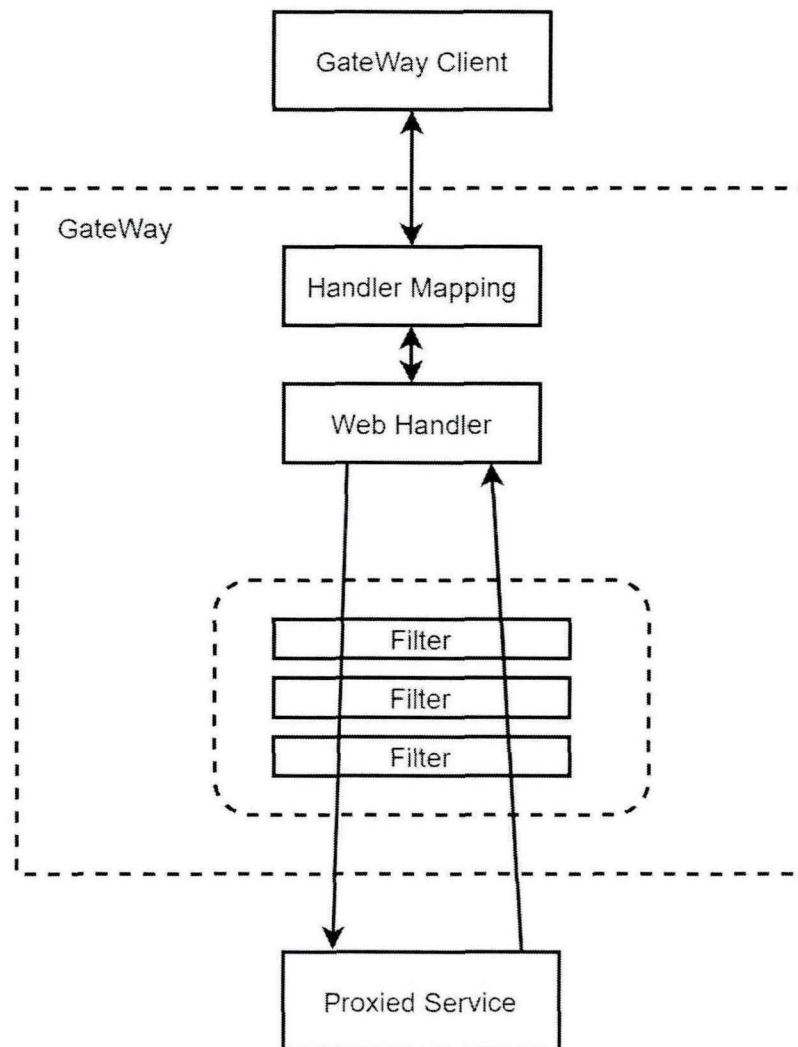


图 5-2 Gateway 的工作原理

而对于 Gateway 的使用，具体的操作步骤如下：

1. 修改配置文件，引入 pom 依赖

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
```

2. 编写 yml 文件


```
spring:
  cloud:
    gateway:
      routes: - id: add_request_parameter_route
               uri: https://example.org
               predicates: - Query=baz
               filters: - AddRequestParameter=foo, b
```

5.2 中央认证服务模块的详细设计与实现

中央认证服务主要是用来验证用户身份,通过对身份的判定保证用户能够正常访问系统资源,同时对非法请求进行过滤的一种功能服务。认证所需要的信息就是用户登录所传递的信息,因此登录是该服务的核心功能,接下来将从单点登录、社交登录和用户注册几方面对中央认证服务进行设计。

5.2.1 单点登录的设计与实现

对于微服务架构的系统来说,服务的部署可能从物理上就被分隔,保存在不同的机器上,服务的资源不可能肆意的被访问,因此服务之间的调用存在访问验证问题。对于面向用户的系统来说,用户发起的请求会传递到服务器获取系统资源,而系统资源又会通过浏览器展示给用户,此时用户与服务器之间必然存在着跨域请求的问题。系统在设计上采用 Nginx 反向代理的方式和添加响应头的方式进行解决。对于用户的一次访问,系统要返回给用户数据需要调用大量的服务进行数据整合,才能返回正确的信息。而每一次服务的转换都需要经过验证,这样不仅对系统造成很大的压力,同时每次验证所花的时间累计将大大提高一次调用的响应时间,会造成用户的一次点击,系统将长时间无响应的尴尬局面。因此为了解决服务之间多次验证的问题,系统将用户的登录功能单独剥开形成认证服务。使用单点登录的方式,让用户只需一次登录,通过携带验证信息,就可以访问所有互信的其他服务,既保证了申请服务资源要经过的验证阶段,又大大降低了验证时间,提高系统的响应速度,也给用户节省了多次登录的步骤。单点登录的 UML 时序图如图 5-3 所示。当用户要访问系统服务时当前服务会判断用户是否携带认证信息,即参数中是否有 token 或许浏览器中的 session 又登录过的信息,如果用户是第一次访问或者是未登录状态,那么该服务就无法对用户的请求发起响应,并将用户指引到中央认证服务进行登录校验。此时用户回到了登录界面,用户会选择自己想要的方式进行登录,在输入账号密码提交后,认证服务会相关的信息进行校验,在通过的前提下,会生成一个由用户信息组成的令牌返回

给请求方，同时系统还会将登录信息保存在 cookie 中。浏览器收到 token 后会保存下来，并将页面跳转到用户第一次想要访问的服务界面并作出响应。当用户想要访问其他服务时，同样服务会验证浏览器是否携带了 token 信息，由于用户已经登录，浏览器保存了 token，会让浏览器携带的 token 和系统保存的 cookie 信息进行比对，通过后会直接允许用户访问服务，同时服务会记录当前的 session 信息，保持用户的登录态，这样用户在一段时间内再次访问该服务时，系统将直接响应用户数据资源，大大节省了中央认证服务器的压力以及响应时间，提高了用户的体验。

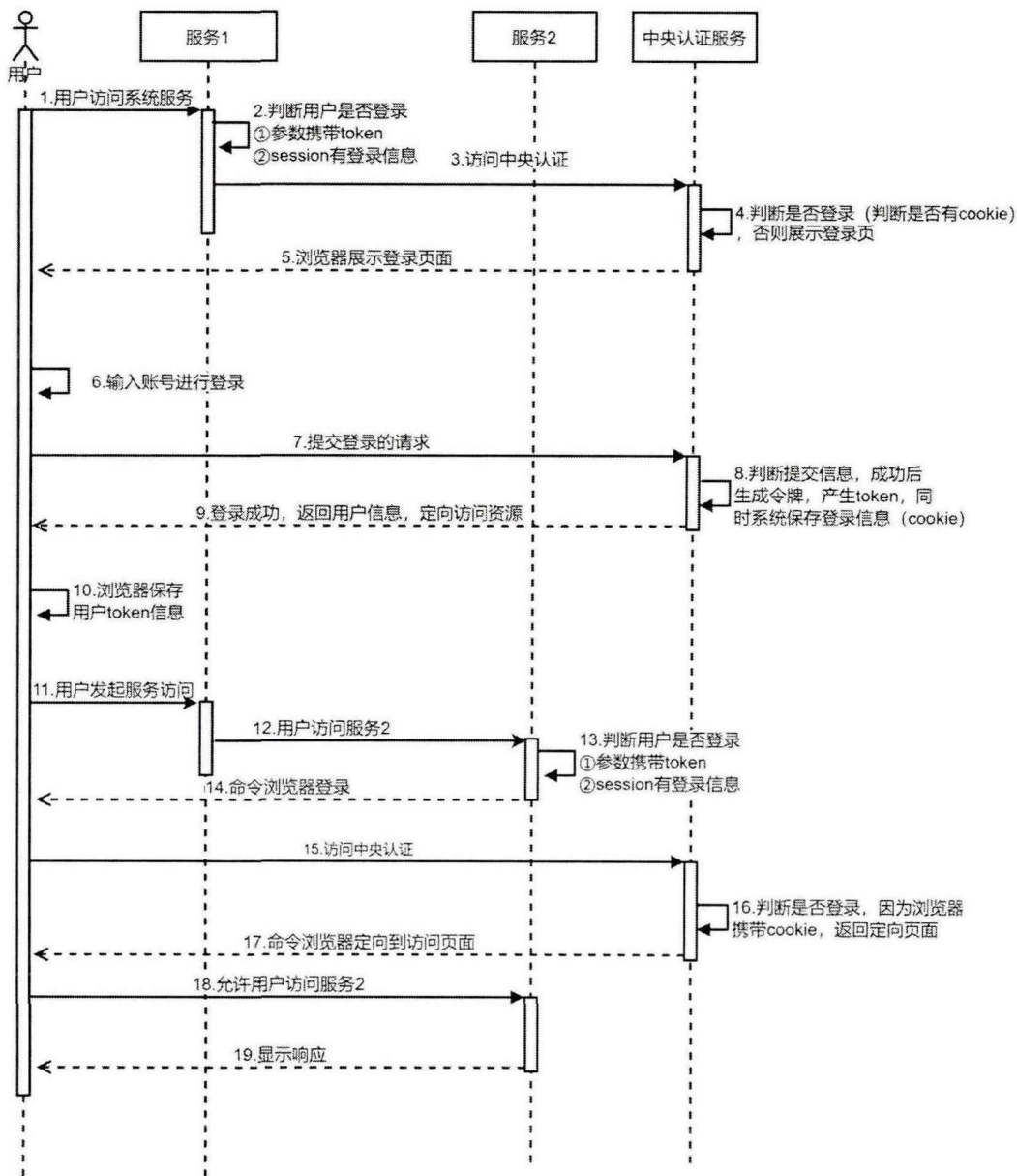


图 5-3 单点登录 UML 时序图

5.2.2 社交登录的设计与实现

上面我们讲到，用户登录使用户访问系统资源的前提，而用户登录的方式可

能有多种方式。社交登录同样是用户喜欢的一种登录方式，系统为了满足用户的需要，这里使用微博登录的方式让用户可以选择社交登录。社交登录的UML时序图如图5-4所示，社交登录使用 OAuth2.0^[20]协议进行授权登录^[21]。第三方网站在征求用户的同意后就可以进行登录。首先登录界面展示给用户登录的方式，用户选择第三方登录后会跳转到第三方授权界面，之后就会通过第三方的系统对输入的信息进行验证，同时为了验证用户的身份，将用户信息保存在自己的系统进行维护与验证，会判断此社交帐号是否是第一次使用。如果是第一次使用，还会让用户进行注册，与相关的社交帐号进行绑定。之后用户再使用社交账号进行后，系统将通过绑定的信息直接定位到用户数据。

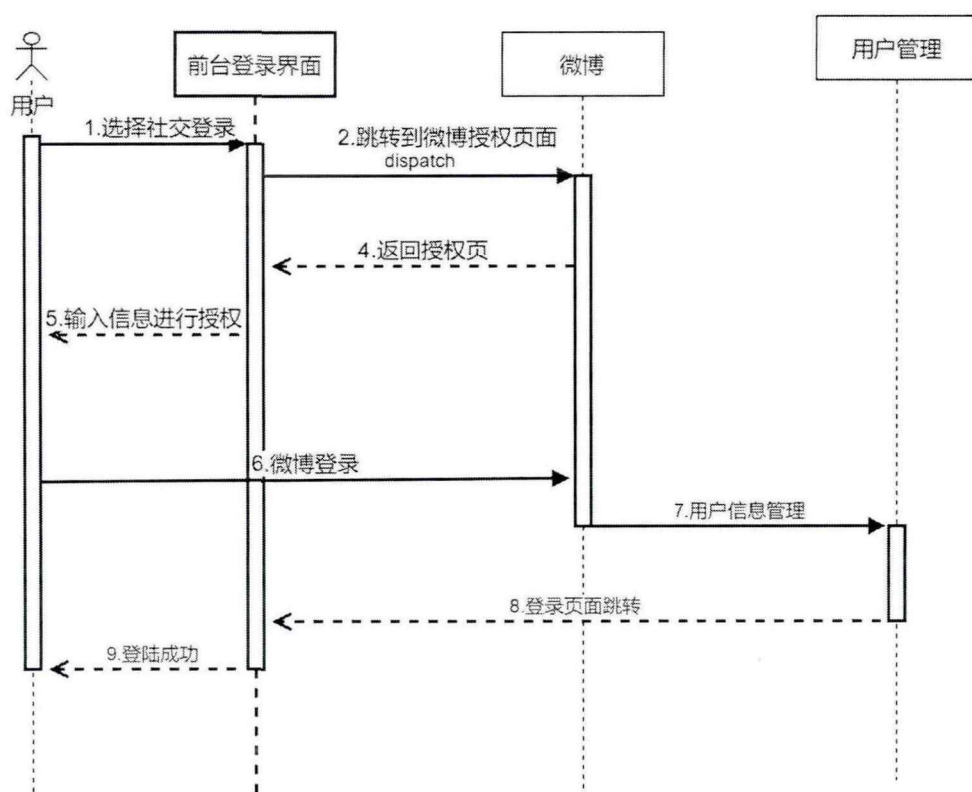


图 5-4 社交登录 UML 时序图

5.2.3 用户注册的设计与实现

用户注册的UML时序图如图5-5所示。对于商城系统来说希望能够有更多的用户能够参与到商品购买当中，而对于新用户就需要注册相关的账号信息保存在系统中，便于系统对用户进行维护，同时只有是注册过的用户才能够使用系统更多的功能，完成购买的流程。在注册时用户必须要输入自己的手机号码作为自己的标识，用户输入手机号码后点击获取验证码，系统就会根据用户输入的手机号码发送相应的验证码，只有当验证码和手机号码相对应的时候用户才能够完成注册。验证码的发送方式是通过接入阿里云的短信验证码的服务进行发送。当然，

用户在点击获取验证码后可能因为手误或者其他原因在短时间内对获取验证码进行多次的点击，这里要使用防刷技术保证短时间内只给用户发送一次验证码，避免造成资源的浪费以及校验的误差，同时要对密码进行加密处理，防止信息泄露。用户注册的实际运行截图如图 5-6 所示。

用户注册部分的代码如下所示。

```
public MemberEntity register(MemberRegVo member){
    checkExistPhone(member.getPhone());
    checkExistUserName(member.getUsername());
    PasswordEncoder passwordEncoder=new PasswordEncoder();
    String encode = passwordEncoder.encode(member.getPassword());
    MemberEntity memberEntity=new MemberEntity()
        .setNickname("加密")
        .setUsername(member.getUsername())
        .setPassword(encode)
        .setMobile(member.getPhone());
    this.baseMapper.insert(memberEntity);
    return memberEntity;
}
```

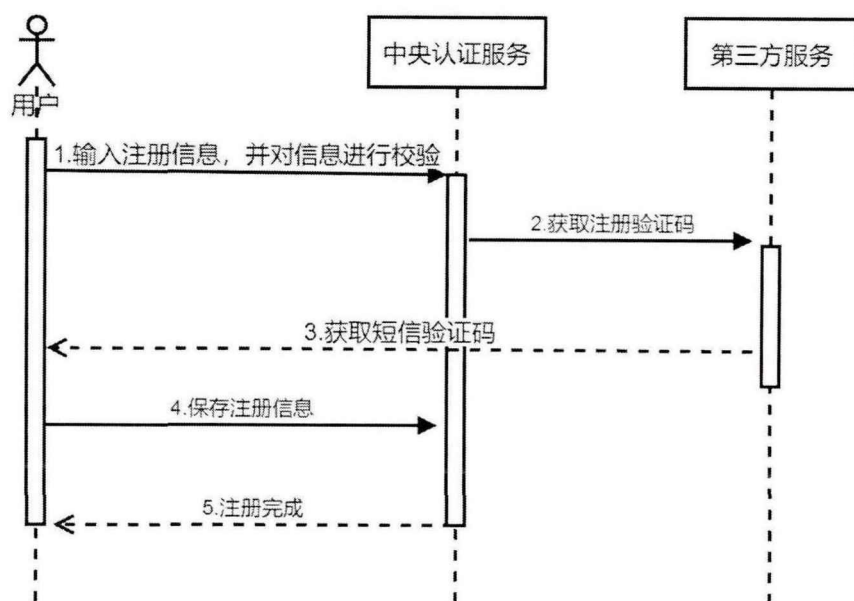


图 5-5 用户注册的 UML 时序图

用户名:

登录密码:

确认密码:

手机号:

短信验证码:

完成注册

图 5-6 用户注册的界面图

5.3 秒杀服务模块的详细设计与实现

秒杀服务作为系统的用户增长服务，旨在通过降低商品价格吸引用户下单，不仅可以通过秒杀活动清理库存，还可以提高用户活跃度进而提高用户下单成功率。秒杀活动的瞬时并发较高，对于系统的资源有着较高的要求。而秒杀不是用户参与网购的必须流程，并且秒杀服务由于负载太高也容易出现问题。因此将秒杀服务作为单独运行的服务，通过限流、异步和缓存策略进行维护。秒杀服务的基础架构图如图 5-7 所示。对于秒杀服务来说，用户所需要的不是当前的商品详情，而是商品的数量是否能够发起秒杀。因此对于用户所看到的秒杀商品，除了特定的可变化的数量外，其他不可变的静态资源都不存放在服务器，而是保存在云服务上。Nginx 会通过用户发起请求的路径通过动静分离的方式路由到不同服务器。商品的图片等信息保存在阿里云上，请求商品库存的请求会被 Nginx 负载均衡的方式打在网关上，请求首先访问的是 Redis 缓存中的数据，由于 Redis 中的数据是热点数据，因此可能会造成缓存击穿，这里使用了加锁的方式进行解决。将商品库存的数量看作是相同数量的锁，每一次对库存的减少都是对当前抢购的数目的加锁，只有当用户获取到锁成功是，才判定用户抢到了商品。系统同时会发送消息队列的消息去轮询当前秒杀的状态，如果用户没有付款或者到达锁的失效时间，系统会将锁定的库存解锁。对于消息队列消息来说，一定要保证消息的可靠性。消息在发送的过程中可能会由于网络波动等其他原因造成消息未传达到消息队列，或者消息到达队列但是消息还没有消费就丢失。对于第一种情况，首先做好消息的日志记录，当消息队列收到消息时就要返回一个确认消息，如果消息的发送方没有收到确认消息，那么就要做好重发的机制，保证消息的正确送达。对于第二种情况，消息要通过落盘操作以防丢失。接下来将从秒杀活动的创建以

及秒杀下单两方面对秒杀服务进行设计。

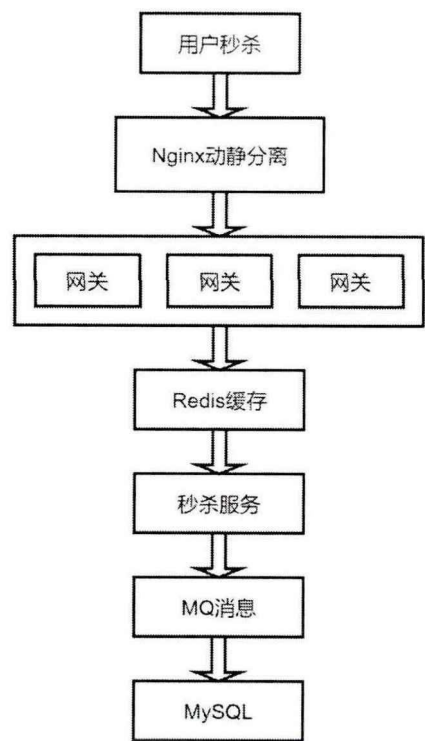


图 5-7 秒杀系统的基础架构图

5.3.1 秒杀活动的创建的设计与实现

秒杀活动的创建的 UML 时序图如图 5-8 所示。秒杀活动的创建是由管理员进行创建，管理员在后台进行活动创建时首先输入活动的名称、活动开始时间和结束时间。当活动创建完毕后，就要为活动创建相关联的商品，输入相关商品的信息后就将商品与活动绑定，接着将秒杀的商品缓存到 Redis 上，因为秒杀的并发很高，而对于 MySQL 来说，单秒内是无法处理更多的请求，同时如果 MySQL 崩溃将会对整个系统造成巨大的影响。此时秒杀活动创建完成。秒杀活动是一种定时任务，每一次活动都不可能由管理员去开启，需要设置一个固定的时间，活动在时间到达的时候会自动开启。对于定时任务的设计采用 Spring 的 schedule 表达式结合 RabbitMQ 的延时队列和死信 Exchange 的方式进行设计。因为当前微服务是分布式系统，而当前的秒杀服务可能会部署到多个节点，因此多个节点的定时任务可能会同时开启，造成数据混乱。这里使用了 ElasticJob 的方式将任务进行拆分，使多个任务并发的去执行，这样既能解决单点问题，又可防止数据混乱。

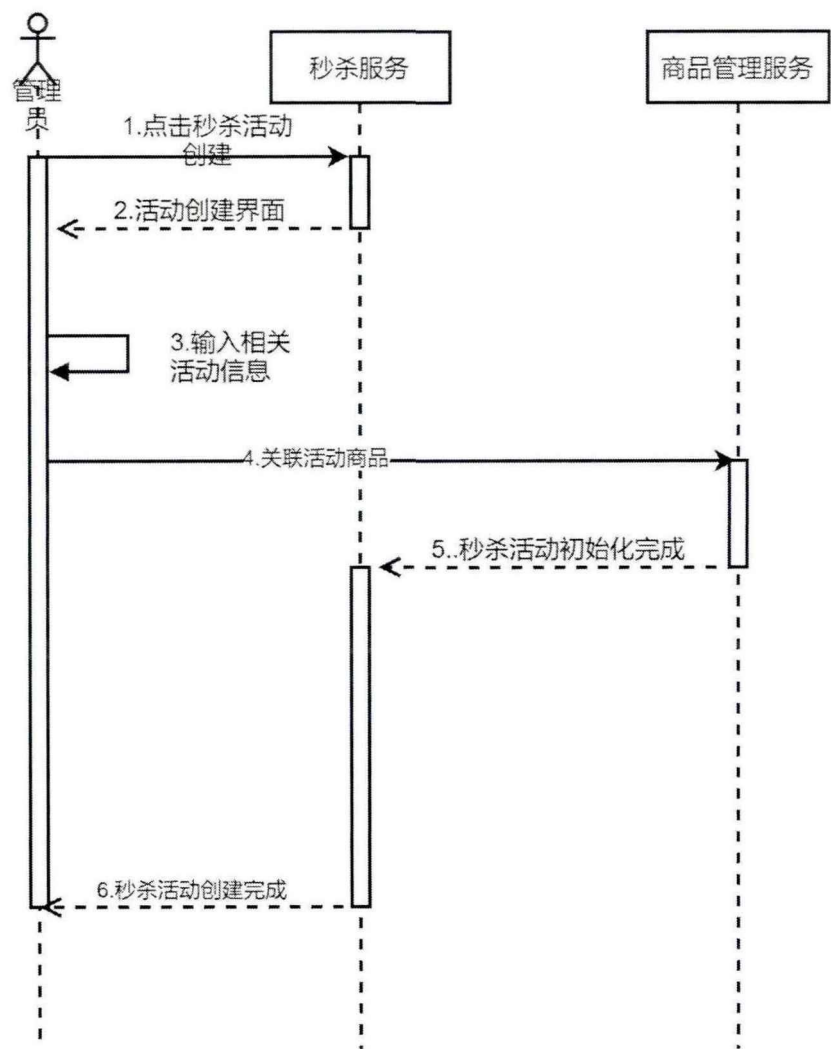


图 5-8 秒杀活动创建的 UML 时序图

5.3.2 秒杀下单的设计与实现

用户秒杀的 UML 时序图如图 5-9 所示。用户在首页查看到秒杀活动后，只要秒杀截止时间未截至或者秒杀商品还有库存，用户就可以参与商品的购买。用户点击抢购商品后，系统先判定用户是否登录，如果用户未登录将不允许用户参与此次秒杀，登录成功后将对用户的信息的合法性进行验证，包括幂等性、登录信息有效性等，核对完成后用户才能下单。此时用户查看的商品数量是 redis 中的商品数量，用户在点击购买后先预减 redis 中的库存，然后创建订单，同时发送消息给订单服务，指定订单服务继续进行订单的创建，订单创建完成后就可以进入地址确认和支付确认页，订单完成。用户秒杀下单入口的系统运行截图如图 5-10 所示。

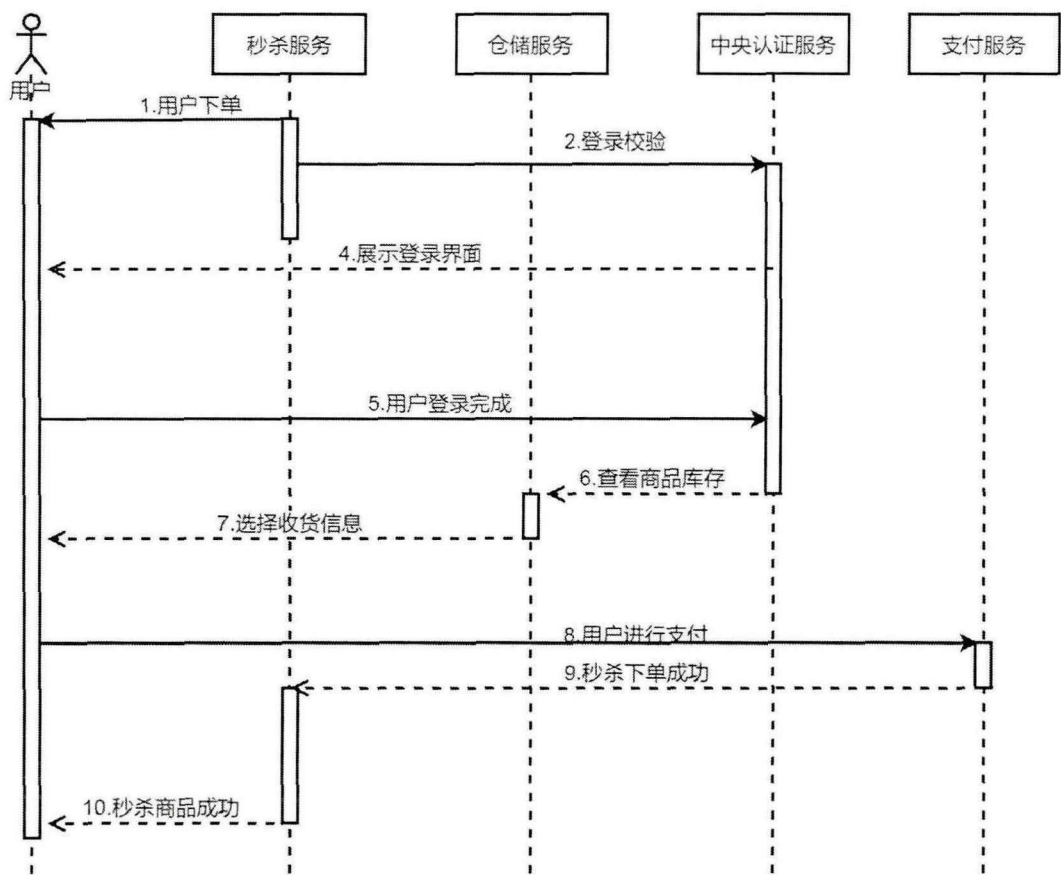


图 5-9 秒杀下单的 UML 时序图



图 5-10 秒杀下单入口界面图

在秒杀活动中，数据保存在缓存中。在秒杀活动中，最重要的就是幂等性的保证，在这里幂等性就是当用户点击秒杀下单时，如果这时候出现了网络波动导致用户无法及时跳转下单界面，这时候用户多次点击下单，最终跳转到下单界面时不会因为用户的多次点击而产生多下单的情况。由于秒杀系统中用户访问量很多而商品很少，幂等性的保证就保证了商品不会被多次购买，幂等性保证的部分代码展示如下：

```
public void uploadSeckillSkuLatest3Days(){
    log.info("商品信息");
    RLock lock = redissonClient.getLock(upload_lock);
    lock.lock(10, TimeUnit.SECONDS);
    try{
        seckillService.uploadSeckillSkuLatest3Days();
    }finally { lock.unlock();}}
```

5.4 订单管理服务模块的详细设计与实现

订单管理服务是电商系统的枢纽,系统的所有功能都是为了用户能够更好的下单,订单的构成需要很多其他信息共同完成。如用户信息包括用户的账号信息和收货人相关信息,一个用户可以创建很多地址,因此一个订单对应一个收货地址。订单的状态是当前订单所进行的环节,包括待付款、已付款、已收货、已完成和售后中。支付信息获取支付流水号、支付方式和支付金额。优惠信息记录用户参与的优惠活动,包括优惠促销活动。商品信息从商品库中获取商品的 SKU 信息、图片、名称、属性规格、商品单价、商户信息等,从用户下单行为记录的用户下单数量,商品合计价格等。接下来将从立即下单和订单退款两个方面对订单服务进行详细的设计。

5.4.1 立即下单的设计与实现

用户下单的 UML 时序图如图 5-11 所示。首先商品管理服务会展示给用户商品的详情,在用户查看商品详情后会进行下单,用户下单是会携带自己的个人信息,此时优惠服务先根据用户的信息查看用户是否有当前购买的商品是否有优惠,并携带优惠信息去访问仓储服务。仓储服务查看当前用户下单的商品库存是否充足,如果库存充足,就按照用户下单数量将库存中相同的商品数量锁定,此时被锁定的商品将不会被其他用户购买,此时仓储服务携带锁定的数目请求订单服务进行下单,订单创建前需要预览订单,选择收货信息。订单服务生成订单完成后就返回支付界面给用户,提示用户进行支付,此时订单的流转状态变成待支付状态。用户将支付结果返回给支付服务,此时订单的流转状态将根据用户的支付结果进行变更,如果用户支付失败或者超时未支付,两种情况下订单都会取消订单,而超时情况是系统自动关闭订单,所以在订单支付的响应机制上面要做支付的限时处理,尤其是在前面说的下单减库存的情形下面,可以保证快速的释放库存。另外需要处理的是促销优惠中使用的优惠券,进行相应补回给用户。此时告知仓储服务将库存解锁,商品数量回增,订单状态流转为未支付。如果此时用户支付成功,此时订单入库,提示仓储服务进行库存的扣减,此时订单状态流转

为已支付。等用户签收商品后，订单状态流转为已签收，此时用户的下单流程基本结束。用户下单结算的系统运行截图如图 5-12 所示。

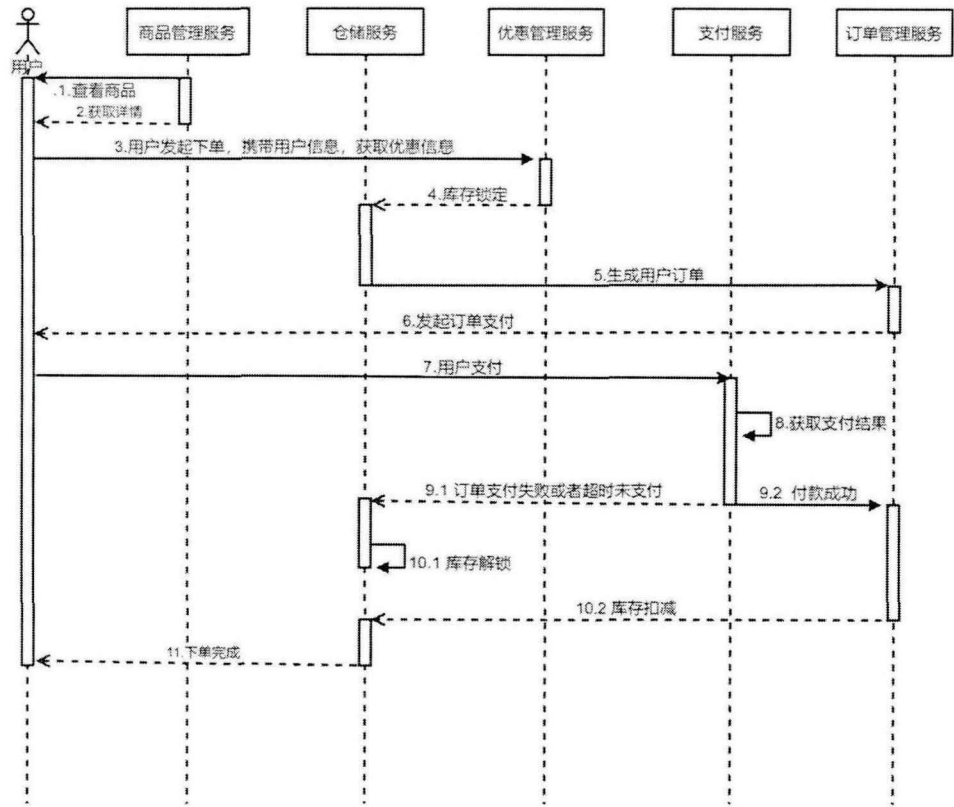


图 5-11 立即下单的 UML 时序图



图 5-12 下单结算的界面图

5.4.2 订单退款的设计与实现

对于商城系统来说，售卖的商品种类繁多，用户量多，很大概率用户因为各

种问题而产生退回退款的行为。因此要增加退货退款的功能来满足用户的需求，同时保证用户的合法权益。此时用户就可以在订单界面点击退货退款来进行退货，当订单服务收到用户的退款请求后就会通知用户寄回商品。之后仓储服务收到了用户寄来的货物后会对商品进行检查，确认后更新库存，将商品入库，此时订单状态流转为退款中，并向支付服务发起退款请求。支付服务收到退款请求后根据记录的账号信息进行原路退款，此时订单结束。用户退货退款的 UML 时序图如图 5-13 所示。

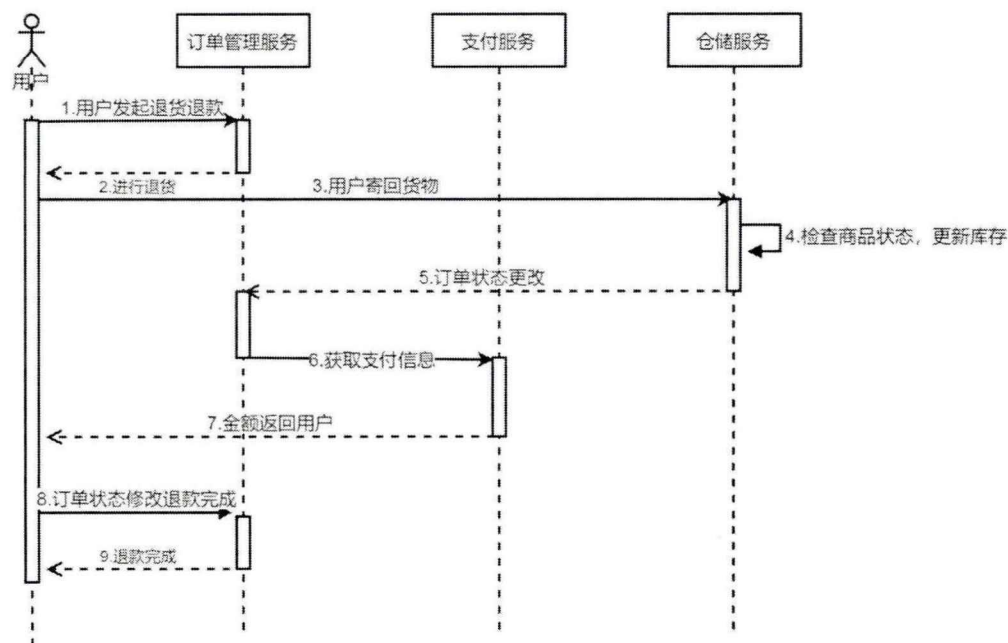


图 5-13 订单退款的 UML 时序图

5.5 支付服务模块的详细设计与实现

用户支付的 UML 时序图如图 5-14 所示。支付服务是为用户付款提供服务，当用户下单时在订单服务创建新的订单，订单服务会根据订单的支付金额向支付服务发起支付请求，之后会查看到自己的订单以及支付类型选择，这里使用了支付宝的支付方式。支付请求会将支付宝的登录界面展示给用户，用户在输入账号密码登录支付宝^[22]并进行支付，支付服务收到用户的付款后会将付款信息告知订单服务，此时支付完成。当用户发起退款时，会从订单服务发起退款操作，订单服务会将用户的信息和订单信息传递给支付服务，支付服务会根据订单的信息找到付款账号，并将金额原路返回给用户，此时用户收到退款。用户支付成功的系统运行截图如图 5-15 所示。

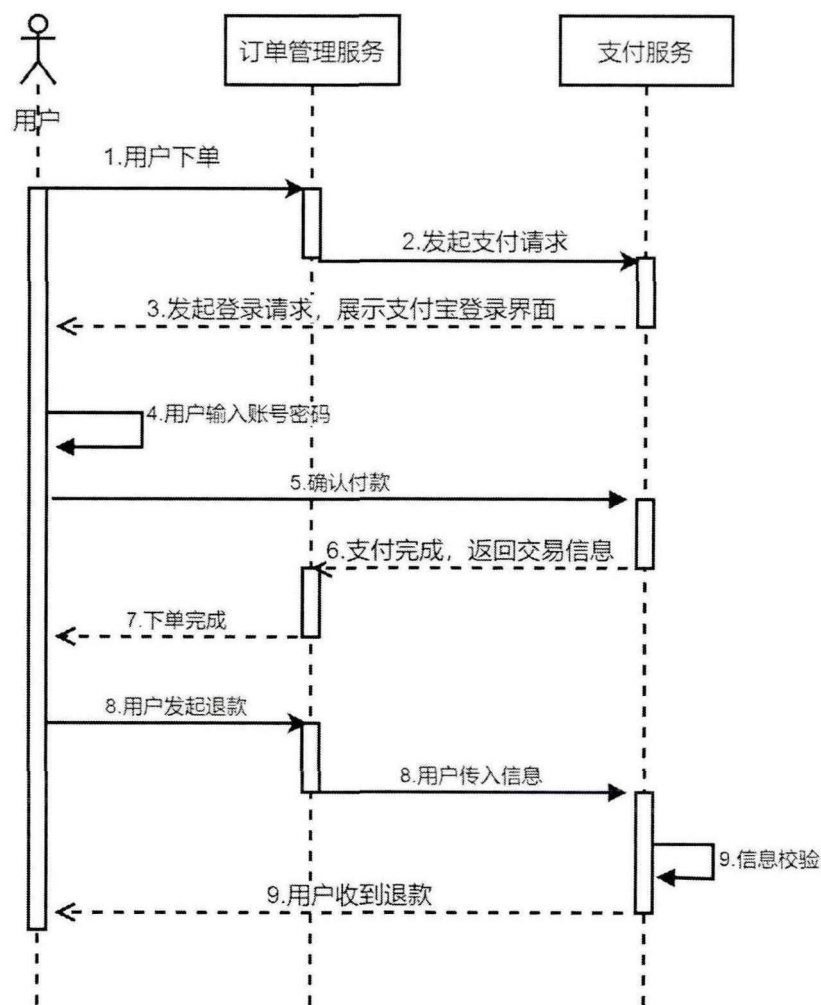


图 5-14 用户支付的 UML 时序图



图 5-15 支付成功的界面图

5.6 商品管理服务模块的详细设计与实现

商品管理服务的 UML 时序图如图 5-16 所示。商品管理服务主要是用来管理商品的基本详情信息。管理员在输入相关的信息，然后判断输入的信息是否符合标准，如果符合，就可以进行修改，否则就要重新输入相关信息。如果要删除其中某些属性，可直接选择属性进行删除，同时属性的列表会在管理员点击当前

功能就会被显示，管理员也可以通过搜索的方式选取属性的特性值进行展示。商品品牌管理的系统运行截图如图 5-17 所示。

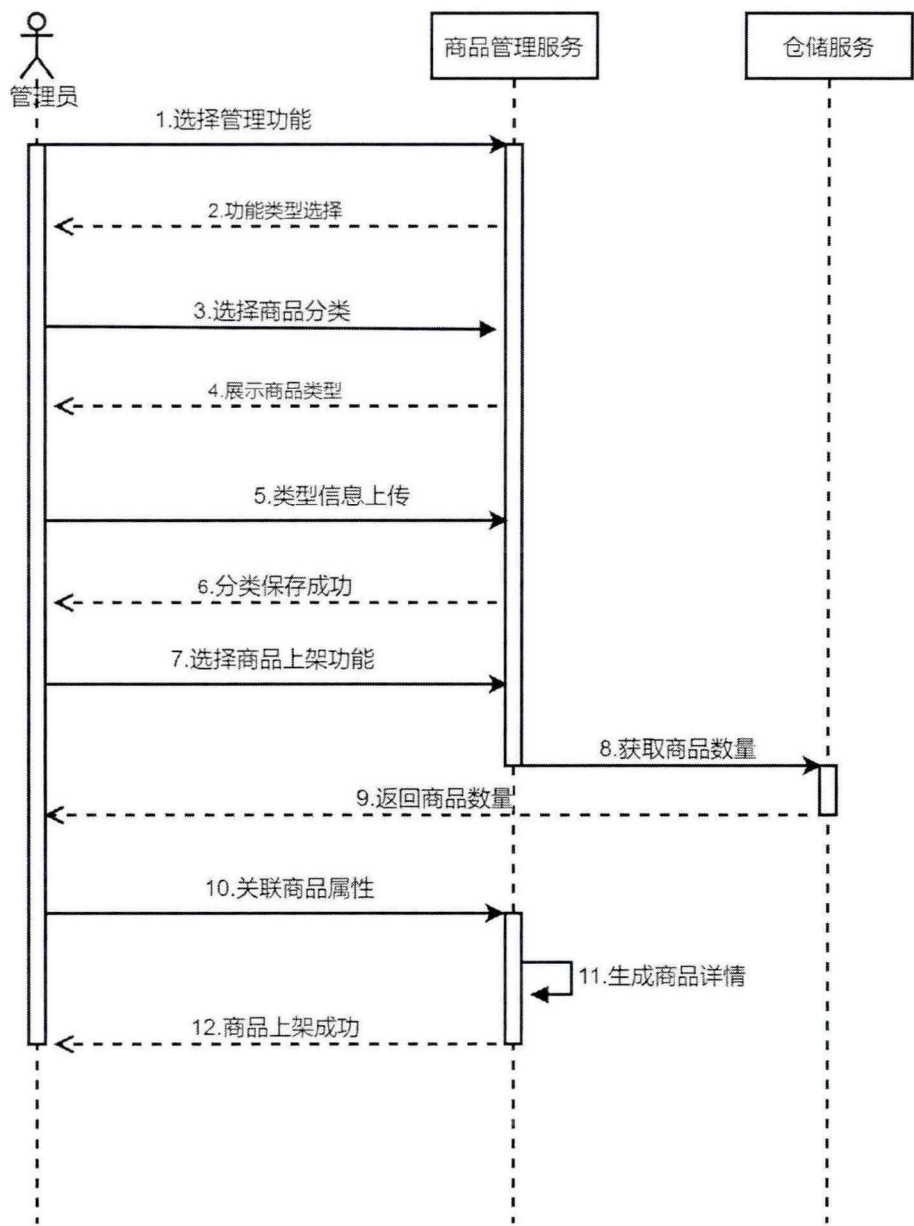


图 5-16 商品管理服务的 UML 时序图



图 5-17 商品品牌管理的界面图

5.7 本章小结

本章在介绍了相关微服务的组件设计之后,主要对中央认证服务、秒杀服务、订单管理服务、支付服务、商品管理服务进行了详细的描述。其中中央认证服务使用单点登录 UML 时序图、社交登录 UML 时序图 and 用户注册的 UML 时序图对用户的登录注册进行了详细的介绍,帮助理解用户登录注册的基本流程;秒杀服务使用秒杀活动创建的 UML 时序图和秒杀下单的 UML 时序图进行介绍,详细介绍了秒杀从创建到被使用的整个过程;订单管理服务使用立即下单 UML 时序图和订单退款 UML 时序图进行了介绍;支付服务使用用户支付 UML 时序图进行详细的介绍;商品管理服务使用商品管理服务 UML 时序图进行详细的介绍。

第六章 系统测试

系统测试是为了保证系统在线上环境的功能可靠性和性能的稳定性。通过对测试结果的分析，判断系统是否符合预期，最终达到正常上线运行使用的目的。接下来将从系统测试方式、系统的功能测试、系统的性能测试、测试结果分析等几个部分对系统的测试进行描述。

6.1 系统测试方法介绍

该系统使用 V 模型的方法，从软件工程的角度对系统进行测试。在在详细设计阶段，将测试用例随着功能的实现方式以及性能的保证方式同时进行设计，保证测试用例能够完整的覆盖系统的功能和性能，这样才能保证测试结果更符合现实。

6.2 系统的功能测试

系统的功能测试是为了检查系统功能是否能够正常使用，保证线上运行时功能不出现问题。对于功能测试，需要将需求分析阶段的功能全都测试完毕以保证系统的所有功能都能满足需求。同时根据系统详细设计阶段同时设计的测试用例，保证用例能够更好覆盖设计上面的缺陷，对于系统的详细设计以及功能点的使用条件设计上起指导作用。系统的功能测试需求分析的具体功能点，使用点击、输入数据等方法进行检测。用户测试内容包括添加地址信息、购物车的相关使用方法与功能、搜索步骤以及搜索内容、付款购买商品等功能。部分测试表如表 6-1 所示，管理员测试内容包括对商品详情信息的修改与添加、存储保存的相关信息的管理步骤、订单相关的查询与展示、活动的创建与商品关联、优惠活动与优惠券的相关信息等。部分测试表如表 6-2 所示。

表 6-1 用户功能测试表

用例编号	测试步骤	测试预期	测试结果	结论
1	点击添加收货地址后，输入基本信息，点击保存。	用户个人信息中出现了新添加的地址	点击个人信息后能够查看到新添加的地址。	通过

(续上表)

用例编号	测试步骤	测试预期	测试结果	结论
2	1.点击添加到购物车。 2.点击商品的加号减号。 3.点击删除。	1.在购物车中找到被保存的商品。 2.商品数量随着用户点击产生变化。 3.购物车中找不到该商品。	1.在购物车中找到被保存的商品。 2.商品数量随着用户点击产生变化。 3.购物车中找不到该商品。	通过
3	1.搜索框中输入华为手机，点击搜索。 2.价格区间输入1000-3000。	1.商品展示区只展示华为手机。 2.商品展示区的华为手机价格都在1000-3000之间。	1.商品展示区只展示华为手机。 2.商品展示区的华为手机价格都在1000-3000之间。	通过
4	1.点击确认购买，展示付款界面。 2.登录支付账号进行付款，付款完成。	点击确认购买后界面跳转支付账号登录界面，输入支付账号后支付完成。	点击确认购买后界面跳转支付账号登录界面，输入支付账号后支付完成。	通过

表 6-2 管理员功能测试表

用例编号	测试步骤	测试预期	测试结果	结论
1	1.点击分类管理 2.点击品牌管理，输入品牌华为点击确定。 3.输入商品的属性信息，点击上架。	1.展示商品的分类。 2.系统显示品牌华为的详细信息。 3.商品上架完成，用户能够查看到商品。	1.展示商品的分类。 2.显示品牌华为的详细信息。 3.商品上架完成，用户能够查看到商品。	通过

(续上表)

用例编号	测试步骤	测试预期	测试结果	结论
2	1. 点击仓库列表。 2. 点击商品的采购单。 3. 点击库存, 输入商品的品牌。	1. 系统展示库存的列表信息。 2. 系统展示商品采购单的具体信息。 3. 显示当前品牌具体商品的库存信息。	1. 系统展示库存的列表信息。 2. 系统展示商品采购单的具体信息。 3. 显示当前品牌具体商品的库存信息。	通过
3	1. 点击订单查看。 2. 输入订单编号, 点击搜索。	1. 显示当前系统的所有订单信息。 2. 显示当前订单编号的订单的具体信息。	1. 显示当前系统的所有订单信息。 2. 显示当前订单编号的订单的具体信息。	通过
4	1. 输入秒杀活动的信息, 点击创建。 2. 点击建立关联, 选中商品, 点击确定。	1. 秒杀活动创建成功, 前台界面展示秒杀活动。 2. 前台的秒杀活动下面展示了相关联的商品。	1. 秒杀活动创建成功, 前台界面展示秒杀活动。 2. 前台的秒杀活动下面展示了相关联的商品。	通过
5	1. 输入优惠活动的信息, 点击创建。 2. 点击优惠活动。 3. 输入优惠券信息, 点击创建。	1. 优惠活动界面展示新建的活动信息。 2. 优惠活动界面展示所有的活动信息。 3. 优惠券列表显示新增的优惠券。	1. 优惠活动界面展示新建的活动信息。 2. 优惠活动界面展示所有的活动信息。 3. 优惠券列表显示新增的优惠券。	通过

6.3 系统的性能测试

系统性能测试的目的是判断当前系统运行时系统的各种性能指标是否满足功能的需要。保证系统在功能可用且满足使用需求的情况下,提高系统的稳定性,提高用户在使用系统时的体验。通过性能的测试,了解系统的瓶颈,为后续对于系统的优化提供可靠的数据支持和方向指引。本节使用了 JMeter 压测工具^[23]对系统的整体性能进行测试。

系统的压力测试如表 6-1 所示,对于系统的商品详情页接口来说,当服务器的请求量达到 2000 时,接口的平均响应时间为 1.5s 左右,接口的响应时间的中位数为 1.4s 左右,其中完成 90%的接口响应时间为 1.8s,完成 95%的接口响应时间为 2.7s,完成 99%的接口响应时间为 3.7s,接口的最大最小响应时间分别为 15ms 和 6.3s,其中 2000 个样本请求全部正常发送。

表 6-1 压力测试表

Label	#样本	平均值	中位数	90%百分位	95%百分位	99%百分位	最小值	最大值	异常 %
HTTP请求	2000	1505	1406	1808	2736	3713	15	6326	0.00%
TOTAL	2000	1505	1406	1808	2736	3713	15	6326	0.00%

系统的普通接口并发量的测试图如图 6-2 所示,在系统的并发量增加的前提下,访问异常数依旧为 0,系统对于并发的产生能够很好的应对,系统的处理时间相对平缓。

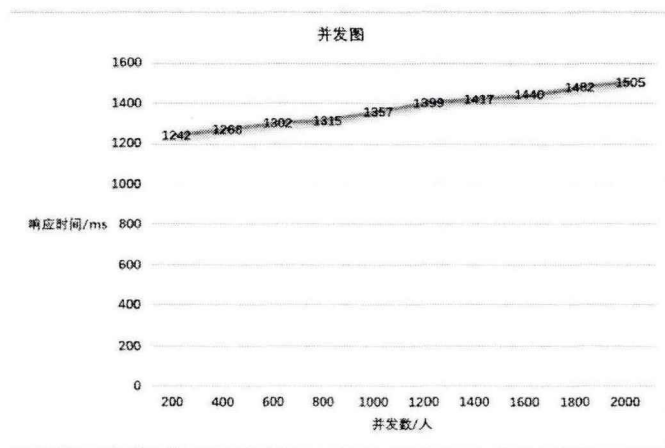


图 6-2 并发量测试图

6.4 测试结果分析

从系统的功能测试结果来看,以不同的功能作为出发点,系统能够满足用户和管理员正常的基本使用,各功能运行正常,不存在异常信息,测试结果达到了预期的要求,各功能表现良好;从系统的性能测试来看,系统能轻松应对上千的请求,同时请求错误率为 0,系统的响应速度也达到了预期的标准,满足了用户

对于系统性能的基本诉求。同时在面对增量的并发请求时，系统能够稳定的应对，保证了系统的高可用性。系统的测试结果符合预期。

6.5 本章小结

本章主要对系统进行测试，在介绍了系统的测试方法之后，系统的功能测试从不同角色的不同功能为出发点，保证了用例的标准与覆盖面的宽泛。在压测基础之上，观看系统在不同压测数据下的不同表现，确认系统的瓶颈，保证了系统处理高并发的能力。并对最后的测试结果进行分析，系统的响应速度满足了使用系统的基本诉求，符合预期的标准。

第七章 结束语

在通过市场的调研之后，在了解了痛点的基础上，本文完成了对基于微服务架构的网上商城系统的详细设计与实现。接下来对本文进行论文的工作总结以及待解决问题的描述，以及对其他的未实现目标的展望。

7.1 论文工作总结

本文分析了商场系统的当前背景以及痛点，为了解决这一痛点，使用微服务的分布式架构设计商城系统，解决了单一架构系统随着需求迭代而带来的种种问题。论述了使用何种技术实现分布式架构，从技术层面映证了痛点解决的可行性，为后续的系统编写提供技术上的支持。通过系统使用角色上的功能出发，制定了系统的需求，完成了系统的总体设计。编码实现了各功能服务，并通过时序图进行具体的介绍。

本文取得的主要成果有：

1. 通过背景的调研，使系统确立微服务的分布式架构进行设计，并描述了相关设计上的技术难点，为最后的编码完成技术支撑。
2. 以用例图的方式使系统功能与使用者一一对应进行需求设计，对系统的总体设计上通过层次架构、界面、数据库以及接口进行设计，了解商场的总体设计由哪些部分组成。
3. 通过对商场系统的具体功能模块进行详细的设计以及编码实现，以中央认证服务等服务为例，对系统的功能进行详细的设计之后编码与测试，等待上线。

本人在该项目中的主要工作有：

1. 完成了网上商城的调研，并对相关的技术进行了查阅。
2. 完成了系统的需求分析，通过用户的角色完成需求的详细设计与确定。
3. 完成了商场系统的软件层次架构设计，同时对系统的数据、系统的界面和系统的接口进行了设计。
4. 完成了微服务组件的设计，以及包括中央认证服务在内的系统详细设计与实现以及在最后的编码实现。
5. 完成了系统测试相关的工作。

7.2 问题和展望

该商城系统采用微服务的架构进行设计,使用前后端分离完成商城业务的整个过程,系统已经上线运行,在运行期间具备处理高并发的能力。但由于个人能力和时间有限,系统仍然存在一定的问题,现对相关的问题描述如下:

1. 系统当前可以满足购物的基本需求,但是随着用户对功能需求的增加,系统的其他功能还有待完善,如用户的拍卖功能、社区团购功能等,通过这些功能能够满足更多人对系统的需求。
2. 当前系统的前端界面只能展示在电脑端,但是随着移动端的继续发展,人们越来越依赖手机进行网上购物,因此后续应该添加移动的 web 访问。
3. 目前前台展示界面不随着用户的不同而不同,缺少用户的定制化服务,后续可以通过对数据进行收集,创建“千人千面”的前台门户。

参考文献

- [1] 李林娟. 中国网络购物现状与发展趋向分析[J]. 中国管理信息化, 2016 (20) 135—136.
- [2] Chen L. Continuous delivery: Huge benefits, but challenges[J]. IEEE software, 2015, 32 (2) :50-54.
- [3] 陈康贤. 大型分布式网站架构设计与实践[M]. 北京: 电子工业出版社, 2014:2.
- [4] 魏茂之. SOA技术架构在企业中的应用[J]. 计算机光盘软件与应用, 2014, 17(09):130-132.
- [5] 刘颖. 基于分布式系统的微服务架构演进[J]. 通讯世界, 2018 (07)97-98.
- [6] SpringCloud [EB/OL]. <https://spring.io/projects/spring-cloud>, 2020.
- [7] Gateway [EB/OL]. <https://cloud.spring.io/spring-cloud-static/spring-cloud-gateway/2.1.3.RELEASE/single/spring-cloud-gateway.html>.
- [8] Sleuth [EB/OL]. <https://cloud.spring.io/spring-cloud-static/spring-cloud-sleuth/2.1.3.RELEASE/single/spring-cloud-sleuth.html>.
- [9] SpringCloudAlibaba [EB/OL]. <https://github.com/alibaba/spring-cloud-alibaba>, 2020.
- [10] NACOS [EB/OL]. <https://www.github.com/alibaba/nacos>, 2018.
- [11] Sentinel [EB/OL]. <https://www.github.com/alibaba/Sentinel>, 2018.
- [12] Elasticsearch [EB/OL]. <https://www.elastic.co/products/elasticsearch>, 2018.
- [13] RabbitMQ [EB/OL]. <https://www.rabbitmq.com/>, 2020.
- [14] 方志朋. 深入理解Spring Cloud与微服务构建[M]. 北京: 人民邮电出版社, 2018.
- [15] 陶辉. 深入理解Nginx[M]. 北京: 机械工业出版社, 2016.
- [16] 廖俊杰, 陶志勇. 微服务API网关的设计及应用[J]. 自动化技术与应用, 2019 (8): 85-88.
- [17] 黄健宏. Redis设计与实现[M]. 北京: 机械工业出版社, 2014.
- [18] 邱祝文. 基于redis的分布式缓存系统架构研究[J]. 网络安全技术与应用, 2014 (10) 52-54.
- [19] 荣艳冬. 关于Mybatis持久化框架的应用研究[J]. 信息安全与技术, 2015 (12): 86—88.
- [20] OAuth2.0 [EB/OL]. <https://oauth.net/2/>, 2018.
- [21] 深入理解OAuth2.0协议 [EB/OL]. <https://www.cnblogs.com/hy18218/p/3584505.html>.
- [22] Java-支付宝支付-支付宝退款 [EB/OL]. <https://www.aliyun.com/jiaocheng/774479.html> 2018.
- [23] Apache. Apache JMeter [EB/OL]. <https://jmeter.apache.org/>.

致 谢

时光荏苒、岁月如梭，转眼间为期三年的硕士研究生生活即将接近尾声。从研究生开始懵懂的我，在老师和同学的帮助下，各方面都取得了很大的进步，在这里我衷心的感谢所有帮助我的人。

首先，我要感谢我的导师，张笑燕导师。张老师严谨的科研作风、渊博的学识、平易近人的品质都让我由衷钦佩。张老师与人和善，笑口常开，让我在与老师的相处中能够放松心态，在学习中获得了很大的收获。同时张老师在论文开题、中期报告以及论文完成等阶段都给了我很大的帮助，对我关于论文的疑惑做出很好的解答。非常幸运能有这段经历，在以后的路上，也会谨记老师的教诲。在此，衷心地感谢张老师。

其次感谢我的家人们，感谢你们对我的理解、支持和鼓励，是你们让我在遇到困难、遭受挫折时，为我提供了坚强的后盾。非常感谢你们的默默付出以及对我的无限包容，我会永怀一颗感恩之心前行。

另外，感谢实验室同学和室友，当我在学习生活上碰到困难时，他们都会给予帮助。也是他们的鼓励和支持，我才能顺利完成本次论文中的项目。

最后，感谢各位评审老师，在百忙之中抽出宝贵的时间评审我的论文和给出建议。